Digital Image Watermarking

Project source code

A hybrid DWT, DCT and SVD approach

Krishang Vikram Singh

Roll No. B106 Batch C1

B.Tech Computer Engineering, NMIMS MPSTME

Subject: Image and Video Processing

Language: Python 3.13 Main libraries: NumPy, SciPy, scikit-image, PyWavelets, Pillow, pandas, matplotlib

Optional: OpenCV and FFmpeg (video test), blind-watermark (comparison)

Contents

Part 1: Core algorithm **4**
1.1 watermark_core.py 4
1.2 metrics.py 5
1.3 attacks.py 6
1.4 logo.py 7
1.5 image_io.py 8
Part 2: Dataset **10**
2.1 dataset_manifest.py 10
2.2 build_dataset.py 12
Part 3: Experiments **16**
3.1 run_experiment.py 16
3.2 run_batch_experiment.py 18
3.3 video_demo.py 21
3.4 exp_jpeg_truncation.py 23
3.5 exp_principal_component_variant.py 24
Part 4: Analysis and benchmarks **25**
4.1 analysis.py 25
4.2 analysis_tech.py 30
4.3 benchmark_tech.py 31
4.4 make_diagram.py 38
Part 5: Tests **40**
5.1 conftest.py 40
5.2 test_watermark_core.py 40
5.3 test_metrics.py 41
5.4 test_attacks.py 42
5.5 test_image_io.py 42
5.6 test_dataset_and_results.py 43
Part 6: Configuration **45**
6.1 config.json 45
6.2 requirements.txt 45

How the project is organised

The code is grouped into six parts. The core algorithm is small and has no dependency on the other parts. The dataset builder prepares the images, the experiment scripts run the algorithm on them and record results, and the analysis scripts turn the recorded results into tables and figures. Tests check the algorithm, the loaders and the completeness of the results.

File	Part	Purpose
watermark_core.py	1. Core algorithm	Hybrid DWT, DCT and SVD embedding and extraction
metrics.py	1. Core algorithm	PSNR, SSIM, MSE, NC, ZNCC and BER
attacks.py	1. Core algorithm	Fourteen attacks plus the unattacked reference
logo.py	1. Core algorithm	Procedural 64x64 binary watermark logo
image_io.py	1. Core algorithm	Safe image loading and standardisation
dataset_manifest.py	2. Dataset	List of source images, licences and categories
build_dataset.py	2. Dataset	Downloads, standardises and records the 100 images
run_experiment.py	3. Experiments	Original two image experiment
run_batch_experiment.py	3. Experiments	Batch experiment on 100 images
video_demo.py	3. Experiments	Per frame video test
exp_jpeg_truncation.py	3. Experiments	Truncation versus rounding before JPEG
exp_principal_component_variant.py	3. Experiments	Principal component embedding variant
analysis.py	4. Analysis and benchmarks	Tables and figures for the main results
analysis_tech.py	4. Analysis and benchmarks	Figures for the technology comparison
benchmark_tech.py	4. Analysis and benchmarks	Library and scheme benchmarks
make_diagram.py	4. Analysis and benchmarks	Pipeline diagrams
conftest.py	5. Tests	Test configuration
test_watermark_core.py	5. Tests	Tests for embedding and extraction
test_metrics.py	5. Tests	Tests for the metrics
test_attacks.py	5. Tests	Tests for the attacks
test_image_io.py	5. Tests	Tests for image loading
test_dataset_and_results.py	5. Tests	Checks on the dataset and results
config.json	6. Configuration	Experiment settings
requirements.txt	6. Configuration	Python packages used

Order in which the main scripts are run

1. build_dataset.py: downloads and standardises the 100 images and writes the metadata table.
2. run_batch_experiment.py: embeds the watermark, applies the attacks, extracts and records every result.
3. analysis.py: statistics, tables and figures from the recorded results.
4. benchmark_tech.py and analysis_tech.py: library and scheme comparisons.
5. video_demo.py: optional per frame video test.
6. pytest tests: runs the automated tests.

Part 1: Core algorithm

The watermarking method itself: embedding, extraction, metrics, attacks, logo and image loading.

1.1 watermark_core.py

Hybrid DWT, DCT and SVD embedding and extraction. 93 lines.

```
from __future__ import annotations

import numpy as np
import pywt
from scipy.fftpack import dct, idct
from skimage.transform import resize

_SUBBANDS = ("LL", "LH", "HL", "HH")


def dct2(block: np.ndarray) -> np.ndarray:
    return dct(dct(block.T, norm="ortho").T, norm="ortho")


def idct2(block: np.ndarray) -> np.ndarray:
    return idct(idct(block.T, norm="ortho").T, norm="ortho")


def _check_2d(name: str, a: np.ndarray, min_side: int = 8) -> np.ndarray:
    a = np.asarray(a)
    if a.ndim != 2:
        raise ValueError(f"{name} must be a 2-D greyscale array, got shape {a.shape}")
    if min(a.shape) < min_side:
        raise ValueError(f"{name} too small: {a.shape}")
    if not np.issubdtype(a.dtype, np.number):
        raise TypeError(f"{name} must be numeric, got {a.dtype}")
    if not np.isfinite(a).all():
        raise ValueError(f"{name} contains NaN/Inf")
    return a.astype(np.float64)


def embed_watermark(host: np.ndarray, watermark: np.ndarray, alpha: float = 0.10,
                    wavelet: str = "haar", subband: str = "LL"):
    host = _check_2d("host", host)
    watermark = _check_2d("watermark", watermark)
    if not alpha > 0:
        raise ValueError(f"alpha must be > 0, got {alpha}")
    if subband not in _SUBBANDS:
        raise ValueError(f"subband must be one of {_SUBBANDS}, got {subband!r}")

    LL, (LH, HL, HH) = pywt.dwt2(host, wavelet)
    subbands = {"LL": LL, "LH": LH, "HL": HL, "HH": HH}
    target = subbands[subband]

    wm_resized = resize(watermark, target.shape, anti_aliasing=True, preserve_range=True)

    target_dct = dct2(target)

    U, S, Vt = np.linalg.svd(target_dct, full_matrices=False)
    Uw, Sw, Vtw = np.linalg.svd(wm_resized, full_matrices=False)

    S_mod = S + alpha * Sw

    target_dct_mod = (U * S_mod) @ Vt
    target_mod = idct2(target_dct_mod)

    subbands[subband] = target_mod
    watermarked = pywt.idwt2((subbands["LL"], (subbands["LH"], subbands["HL"], subbands["HH"])),
wavelet)

    watermarked = watermarked[:host.shape[0], :host.shape[1]]
    watermarked = np.clip(watermarked, 0, 255)

    key = {
        "U": U, "Vt": Vt, "S_original": S,
        "Uw": Uw, "Vtw": Vtw,
        "alpha": alpha, "wavelet": wavelet, "subband": subband,
        "wm_shape": watermark.shape, "target_shape": target.shape,
```

```
    }
    return watermarked, key


def extract_watermark(possibly_attacked: np.ndarray, key: dict) -> np.ndarray:
    img = _check_2d("image", possibly_attacked)
    wavelet = key["wavelet"]
    subband = key["subband"]

    LL, (LH, HL, HH) = pywt.dwt2(img, wavelet)
    subbands = {"LL": LL, "LH": LH, "HL": HL, "HH": HH}
    target = subbands[subband]

    target_shape = key["target_shape"]
    if target.shape != target_shape:
        target = resize(target, target_shape, anti_aliasing=True, preserve_range=True)

    target_dct = dct2(target)
    _, S_att, _ = np.linalg.svd(target_dct, full_matrices=False)

    Sw_extracted = (S_att - key["S_original"]) / key["alpha"]
    watermark_extracted = (key["Uw"] * Sw_extracted) @ key["Vtw"]

    watermark_extracted = resize(watermark_extracted, key["wm_shape"],
                                 anti_aliasing=True, preserve_range=True)
    return np.clip(watermark_extracted, 0, 255)
```

1.2 metrics.py

PSNR, SSIM, MSE, NC, ZNCC and BER. 63 lines.

```
import numpy as np
from skimage.metrics import peak_signal_noise_ratio as _psnr
from skimage.metrics import structural_similarity as _ssim

PSNR_CAP_DB = 100.0


def _prep(a: np.ndarray, b: np.ndarray):
    a = np.clip(np.asarray(a, dtype=np.float64), 0, 255)
    b = np.clip(np.asarray(b, dtype=np.float64), 0, 255)
    if a.shape != b.shape:
        raise ValueError(f"shape mismatch: {a.shape} vs {b.shape}")
    return a, b


def mse(original: np.ndarray, test: np.ndarray) -> float:
    a, b = _prep(original, test)
    return float(np.mean((a - b) ** 2))


def psnr(original: np.ndarray, test: np.ndarray) -> float:
    a, b = _prep(original, test)
    if np.array_equal(a, b):
        return PSNR_CAP_DB
    return float(min(_psnr(a, b, data_range=255), PSNR_CAP_DB))


def ssim(original: np.ndarray, test: np.ndarray) -> float:
    a, b = _prep(original, test)
    return float(_ssim(a, b, data_range=255))


def normalized_correlation(original_wm: np.ndarray, extracted_wm: np.ndarray) -> float:
    a = np.asarray(original_wm, dtype=np.float64).flatten()
    b = np.asarray(extracted_wm, dtype=np.float64).flatten()
    if a.shape != b.shape:
        raise ValueError(f"shape mismatch: {original_wm.shape} vs {extracted_wm.shape}")
    num = np.sum(a * b)
    den = np.sqrt(np.sum(a ** 2)) * np.sqrt(np.sum(b ** 2))
    if den == 0:
        return 0.0
    return float(num / den)


def zero_normalized_correlation(original_wm: np.ndarray, extracted_wm: np.ndarray) -> float:
    a = np.asarray(original_wm, dtype=np.float64).flatten()
    b = np.asarray(extracted_wm, dtype=np.float64).flatten()
```

```
    if a.shape != b.shape:
        raise ValueError(f"shape mismatch: {original_wm.shape} vs {extracted_wm.shape}")
    a = a - a.mean()
    b = b - b.mean()
    den = np.sqrt(np.sum(a ** 2)) * np.sqrt(np.sum(b ** 2))
    if den == 0:
        return 0.0
    return float(np.sum(a * b) / den)


def bit_error_rate(original_wm: np.ndarray, extracted_wm: np.ndarray, thresh: float = 127.5) ->
float:
    a = (np.asarray(original_wm, dtype=np.float64) > thresh).astype(np.uint8).flatten()
    b = (np.asarray(extracted_wm, dtype=np.float64) > thresh).astype(np.uint8).flatten()
    if a.shape != b.shape:
        raise ValueError(f"shape mismatch: {original_wm.shape} vs {extracted_wm.shape}")
    return float(np.mean(a != b))
```

1.3 attacks.py

Fourteen attacks plus the unattacked reference. 112 lines.

```
from __future__ import annotations

import io

import numpy as np
from PIL import Image
from scipy import ndimage
from skimage.exposure import equalize_hist
from skimage.transform import rotate, resize
from skimage.util import random_noise


def jpeg_compression(img: np.ndarray, quality: int = 50) -> np.ndarray:
    if not 1 <= quality <= 95:
        raise ValueError("quality must be in 1..95")
    im = Image.fromarray(np.clip(np.rint(img), 0, 255).astype(np.uint8))
    buf = io.BytesIO()
    im.save(buf, format="JPEG", quality=quality)
    buf.seek(0)
    return np.array(Image.open(buf).convert("L")).astype(np.float64)


def gaussian_noise(img: np.ndarray, var: float = 0.01, rng=None) -> np.ndarray:
    noisy = random_noise(img / 255.0, mode="gaussian", var=var, rng=rng)
    return noisy * 255.0


def salt_pepper_noise(img: np.ndarray, amount: float = 0.02, rng=None) -> np.ndarray:
    noisy = random_noise(img / 255.0, mode="s&p", amount=amount, rng=rng)
    return noisy * 255.0


def median_filter(img: np.ndarray, size: int = 3) -> np.ndarray:
    return ndimage.median_filter(img, size=size)


def gaussian_blur(img: np.ndarray, sigma: float = 1.5) -> np.ndarray:
    return ndimage.gaussian_filter(img, sigma=sigma)


def rotation_attack(img: np.ndarray, angle: float = 5.0) -> np.ndarray:
    rotated = rotate(img / 255.0, angle, resize=False, mode="edge")
    return rotated * 255.0


def cropping_attack(img: np.ndarray, frac: float = 0.25) -> np.ndarray:
    h, w = img.shape
    ch, cw = int(h * frac / 2), int(w * frac / 2)
    out = img.copy()
    out[:ch, :] = 0
    out[h - ch:, :] = 0
    out[:, :cw] = 0
    out[:, w - cw:] = 0
    return out
```

```
def scaling_attack(img: np.ndarray, factor: float = 0.5) -> np.ndarray:
    h, w = img.shape
    small = resize(img, (int(h * factor), int(w * factor)), anti_aliasing=True, preserve_range=True)
    back = resize(small, (h, w), anti_aliasing=True, preserve_range=True)
    return back


def histogram_equalization(img: np.ndarray) -> np.ndarray:
    eq = equalize_hist(img / 255.0)
    return eq * 255.0


def sharpening_attack(img: np.ndarray, amount: float = 1.0) -> np.ndarray:
    blurred = ndimage.gaussian_filter(img, sigma=1.0)
    sharpened = img + amount * (img - blurred)
    return np.clip(sharpened, 0, 255)


ATTACK_NAMES = [
    "No Attack", "JPEG Q=90", "JPEG Q=50", "JPEG Q=20",
    "Gaussian Noise (var=0.005)", "Gaussian Noise (var=0.02)", "Salt & Pepper (2%)",
    "Median Filter 3x3", "Gaussian Blur (sigma=1.5)", "Rotation 5 deg", "Rotation 15 deg",
    "Cropping 25%", "Scaling 0.5x", "Histogram Equalization", "Sharpening",
]

ATTACK_FAMILY = {
    "No Attack": "none", "JPEG Q=90": "compression", "JPEG Q=50": "compression", "JPEG Q=20":
"compression",
    "Gaussian Noise (var=0.005)": "noise", "Gaussian Noise (var=0.02)": "noise", "Salt & Pepper
(2%)": "noise",
    "Median Filter 3x3": "filtering", "Gaussian Blur (sigma=1.5)": "filtering", "Sharpening":
"filtering",
    "Rotation 5 deg": "geometric", "Rotation 15 deg": "geometric", "Cropping 25%": "geometric",
    "Scaling 0.5x": "geometric", "Histogram Equalization": "global intensity",
}


def make_attack_suite(seed: int = 0) -> dict:
    return {
        "No Attack":                  lambda x: x.copy(),
        "JPEG Q=90":                  lambda x: jpeg_compression(x, 90),
        "JPEG Q=50":                  lambda x: jpeg_compression(x, 50),
        "JPEG Q=20":                  lambda x: jpeg_compression(x, 20),
        "Gaussian Noise (var=0.005)": lambda x: gaussian_noise(x, 0.005, rng=seed),
        "Gaussian Noise (var=0.02)":  lambda x: gaussian_noise(x, 0.02, rng=seed + 1),
        "Salt & Pepper (2%)":         lambda x: salt_pepper_noise(x, 0.02, rng=seed + 2),
        "Median Filter 3x3":          lambda x: median_filter(x, 3),
        "Gaussian Blur (sigma=1.5)":  lambda x: gaussian_blur(x, 1.5),
        "Rotation 5 deg":             lambda x: rotation_attack(x, 5),
        "Rotation 15 deg":            lambda x: rotation_attack(x, 15),
        "Cropping 25%":               lambda x: cropping_attack(x, 0.25),
        "Scaling 0.5x":               lambda x: scaling_attack(x, 0.5),
        "Histogram Equalization":     lambda x: histogram_equalization(x),
        "Sharpening":                 lambda x: sharpening_attack(x, 1.0),
    }


ATTACK_SUITE = make_attack_suite(0)
assert list(ATTACK_SUITE) == ATTACK_NAMES
```

1.4 logo.py

Procedural 64x64 binary watermark logo. 17 lines.

```
import numpy as np
from PIL import Image, ImageDraw, ImageFont


def make_watermark_logo(size=64, text="CYBERION"):
    img = Image.new("L", (size * 4, size), color=0)
    draw = ImageDraw.Draw(img)
    try:
        font = ImageFont.load_default()
    except Exception:
        font = None
    draw.rectangle([0, 0, size * 4 - 1, size - 1], outline=255, width=3)
    draw.text((10, size // 2 - 6), text, fill=255, font=font)
    img = img.resize((size, size))
```

```
    arr = np.array(img).astype(np.float64)
    arr = (arr > 100).astype(np.float64) * 255.0
    return arr
```

1.5 image_io.py

Safe image loading and standardisation. 86 lines.

```
from __future__ import annotations

import os
from dataclasses import dataclass

import numpy as np
from PIL import Image, UnidentifiedImageError

Image.MAX_IMAGE_PIXELS = 200_000_000


class ImageLoadError(RuntimeError):
    pass


@dataclass
class SourceInfo:
    width: int
    height: int
    mode: str
    fmt: str
    file_size: int


def _to_uint8_rgbish(im: Image.Image) -> Image.Image:
    mode = im.mode
    if mode in ("RGBA", "LA", "PA") or (mode == "P" and "transparency" in im.info):
        im = im.convert("RGBA")
        bg = Image.new("RGBA", im.size, (255, 255, 255, 255))
        return Image.alpha_composite(bg, im).convert("RGB")
    if mode in ("I;16", "I;16B", "I;16L", "I", "F"):
        arr = np.asarray(im, dtype=np.float64)
        lo, hi = float(arr.min()), float(arr.max())
        if hi <= lo:
            raise ImageLoadError("constant-valued high-bit-depth image")
        return Image.fromarray(((arr - lo) / (hi - lo) * 255.0).round().astype(np.uint8), "L")
    if mode == "1":
        return im.convert("L")
    if mode in ("L", "RGB"):
        return im
    return im.convert("RGB")


def load_gray_square(path: str, size: int = 512) -> tuple[np.ndarray, SourceInfo]:
    if not os.path.isfile(path):
        raise ImageLoadError(f"file not found: {path}")
    fsize = os.path.getsize(path)
    if fsize == 0:
        raise ImageLoadError(f"empty file: {path}")
    try:
        with Image.open(path) as im:
            fmt = im.format or ""
            w, h = im.size
            mode = im.mode
            im.load()
            rgb = _to_uint8_rgbish(im)
    except (UnidentifiedImageError, OSError, ValueError, SyntaxError) as exc:
        raise ImageLoadError(f"cannot decode {os.path.basename(path)}: {exc}") from exc
    if min(w, h) < 16:
        raise ImageLoadError(f"image too small ({w}x{h})")

    gray = rgb.convert("L")
    side = min(gray.size)
    left, top = (gray.width - side) // 2, (gray.height - side) // 2
    gray = gray.crop((left, top, left + side, top + side)).resize((size, size), Image.LANCZOS)
    arr = np.asarray(gray, dtype=np.float64)
    return arr, SourceInfo(w, h, mode, fmt, fsize)


def save_gray_png(arr: np.ndarray, path: str) -> None:
```

```
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    Image.fromarray(np.clip(np.rint(arr), 0, 255).astype(np.uint8), "L").save(path)


def verify_png(path: str, expected_shape: tuple[int, int] | None = None) -> bool:
    try:
        with Image.open(path) as im:
            im.load()
            a = np.asarray(im)
    except Exception:
        return False
    if a.ndim != 2 or a.dtype != np.uint8:
        return False
    if expected_shape is not None and a.shape != tuple(expected_shape):
        return False
    return bool(np.isfinite(a).all())
```

Part 2: Dataset

Everything needed to rebuild the 100 image evaluation set.

2.1 dataset_manifest.py

List of source images, licences and categories. 118 lines.

```
OCV = "opencv/opencv/4.x/samples/data/"
OCVX = "opencv/opencv_extra/4.x/testdata/"
OXF = OCVX + "cv/detectors_descriptors_evaluation/images_datasets/"
SKI = "scikit-image/scikit-image/v0.26.0/src/skimage/data/"
THR = "mrdoob/three.js/dev/examples/textures/"
PIL = "python-pillow/Pillow/main/Tests/images/"

_APACHE = "Apache-2.0"
_NOT_DOC = "not individually documented upstream"

REAL_IMAGES = [
    ("ocv_apple",        OCV + "apple.jpg",        "OpenCV samples", _APACHE, _NOT_DOC, "object"),
    ("ocv_baboon",       OCV + "baboon.jpg",       "OpenCV samples", _APACHE, "classic USC-SIPI
'mandrill' test image; " + _NOT_DOC, "animal"),
    ("ocv_board",        OCV + "board.jpg",        "OpenCV samples", _APACHE, _NOT_DOC,
"object_fine_detail"),
    ("ocv_building",     OCV + "building.jpg",     "OpenCV samples", _APACHE, _NOT_DOC,
"urban_architecture"),
    ("ocv_butterfly",    OCV + "butterfly.jpg",    "OpenCV samples", _APACHE, _NOT_DOC, "animal"),
    ("ocv_chicky",       OCV + "chicky_512.png",   "OpenCV samples", _APACHE, _NOT_DOC, "animal"),
    ("ocv_fruits",       OCV + "fruits.jpg",       "OpenCV samples", _APACHE, _NOT_DOC, "object"),
    ("ocv_graf1",        OCV + "graf1.png",        "OpenCV samples", _APACHE, "Oxford affine-regions
'graf' image; research use", "urban_architecture"),
    ("ocv_home",         OCV + "home.jpg",         "OpenCV samples", _APACHE, _NOT_DOC,
"urban_architecture"),
    ("ocv_left01",       OCV + "left01.jpg",       "OpenCV samples", _APACHE, "calibration-pattern
scene (one of 12 near-identical frames; only one kept)", "indoor_person"),
    ("ocv_messi",        OCV + "messi5.jpg",       "OpenCV samples", _APACHE, _NOT_DOC,
"person_sports"),
    ("ocv_orange",       OCV + "orange.jpg",       "OpenCV samples", _APACHE, _NOT_DOC, "object"),
    ("ocv_smarties",     OCV + "smarties.png",     "OpenCV samples", _APACHE, _NOT_DOC, "object"),
    ("ocv_starry_night", OCV + "starry_night.jpg", "OpenCV samples", _APACHE, "reproduction of a
public-domain painting (van Gogh, 1889)", "painting_art"),
    ("ocv_stuff",        OCV + "stuff.jpg",        "OpenCV samples", _APACHE, _NOT_DOC, "object"),
    ("ocv_rubberwhale",  OCV + "rubberwhale1.png", "OpenCV samples", _APACHE, "Middlebury optical-
flow dataset frame; research use", "object"),
    ("ocv_text_motion",  OCV + "text_motion.jpg",  "OpenCV samples", _APACHE, _NOT_DOC,
"document_text"),
    ("ocv_imagetext",    OCV + "imageTextN.png",   "OpenCV samples", _APACHE, _NOT_DOC,
"document_text"),
    ("ocv_basketball",   OCV + "basketball1.png",  "OpenCV samples", _APACHE, "video frame; " +
_NOT_DOC, "indoor_person"),
    ("ocv_blox",         OCV + "blox.jpg",         "OpenCV samples", _APACHE, _NOT_DOC, "object"),
    ("ocv_books",        OCV + "left.jpg",         "OpenCV samples", _APACHE, "left view of a stereo
pair (right view omitted); " + _NOT_DOC, "indoor_object"),
    ("ocv_aloe",         OCV + "aloeL.jpg",        "OpenCV samples", _APACHE, "Middlebury stereo
dataset (left view only); research use", "plant_object"),
    ("ocvx_airplane",    OCVX + "cv/shared/airplane.png",       "OpenCV extra", _APACHE, "classic
USC-SIPI 'airplane (F-16)' test image; " + _NOT_DOC, "aerial_vehicle"),
    ("ocvx_monalisa",    OCVX + "cv/cascadeandhog/images/mona-lisa.png", "OpenCV extra", _APACHE,
"reproduction of a public-domain painting", "painting_art"),
    ("ocvx_shuttle",     OCVX + "dnn/space_shuttle.jpg",        "OpenCV extra", _APACHE, _NOT_DOC,
"aerial_vehicle"),
    ("ocvx_street",      OCVX + "dnn/street.png",               "OpenCV extra", _APACHE, _NOT_DOC,
"urban_architecture"),
    ("ocvx_cones",       OCVX + "cv/stereomatching/datasets/cones/im2.png", "OpenCV extra", _APACHE,
"Middlebury stereo 'cones' (view 2); research use", "indoor_object"),
    ("ocvx_david",       OCVX + "cv/face/david1.jpg",           "OpenCV extra", _APACHE, "face video
frame; " + _NOT_DOC, "person_portrait"),
    ("oxf_bark",         OXF + "bark/img1.png",   "Oxford affine-regions (via OpenCV extra)",
_APACHE, "K. Mikolajczyk et al., Oxford VGG; research use", "texture_natural"),
    ("oxf_bikes",        OXF + "bikes/img1.png",  "Oxford affine-regions (via OpenCV extra)",
_APACHE, "K. Mikolajczyk et al., Oxford VGG; research use", "urban_architecture"),
    ("oxf_boat",         OXF + "boat/img1.png",   "Oxford affine-regions (via OpenCV extra)",
_APACHE, "K. Mikolajczyk et al., Oxford VGG; research use", "urban_architecture"),
    ("oxf_leuven",       OXF + "leuven/img1.png", "Oxford affine-regions (via OpenCV extra)",
_APACHE, "K. Mikolajczyk et al., Oxford VGG; research use", "urban_architecture"),
```

```
    ("oxf_trees",        OXF + "trees/img1.png",  "Oxford affine-regions (via OpenCV extra)",
_APACHE, "K. Mikolajczyk et al., Oxford VGG; research use", "natural_scene"),
    ("oxf_ubc",          OXF + "ubc/img1.png",    "Oxford affine-regions (via OpenCV extra)",
_APACHE, "K. Mikolajczyk et al., Oxford VGG; research use", "urban_architecture"),
    ("oxf_wall",         OXF + "wall/img1.png",   "Oxford affine-regions (via OpenCV extra)",
_APACHE, "K. Mikolajczyk et al., Oxford VGG; research use", "texture_natural"),
    ("dkn_dog",          "pjreddie/darknet/master/data/dog.jpg",     "darknet", "YOLO licence (public
domain)", _NOT_DOC, "animal"),
    ("dkn_eagle",        "pjreddie/darknet/master/data/eagle.jpg",   "darknet", "YOLO licence (public
domain)", _NOT_DOC, "animal"),
    ("dkn_giraffe",      "pjreddie/darknet/master/data/giraffe.jpg", "darknet", "YOLO licence (public
domain)", _NOT_DOC, "animal"),
    ("dkn_horses",       "pjreddie/darknet/master/data/horses.jpg",  "darknet", "YOLO licence (public
domain)", _NOT_DOC, "animal"),
    ("dkn_kite",         "pjreddie/darknet/master/data/kite.jpg",    "darknet", "YOLO licence (public
domain)", _NOT_DOC, "natural_scene"),
    ("dkn_person",       "pjreddie/darknet/master/data/person.jpg",  "darknet", "YOLO licence (public
domain)", _NOT_DOC, "natural_scene"),
    ("dkn_scream",       "pjreddie/darknet/master/data/scream.jpg",  "darknet", "YOLO licence (public
domain)", "reproduction of a painting (Munch, 1893)", "painting_art"),
    ("tv_dog1",          "pytorch/vision/main/gallery/assets/dog1.jpg", "torchvision gallery", "BSD-
3-Clause", _NOT_DOC, "animal"),
    ("tv_dog2",          "pytorch/vision/main/gallery/assets/dog2.jpg", "torchvision gallery", "BSD-
3-Clause", _NOT_DOC, "animal"),
    ("tfm_dogs",         "tensorflow/models/master/research/object_detection/test_images/image1.jpg",
"TensorFlow models", _APACHE, _NOT_DOC, "animal"),
    ("tfm_beach",        "tensorflow/models/master/research/object_detection/test_images/image2.jpg",
"TensorFlow models", _APACHE, _NOT_DOC, "natural_scene"),
    ("sam_dog",          "facebookresearch/segment-anything/main/notebooks/images/dog.jpg",
"segment-anything", _APACHE, _NOT_DOC, "indoor_person"),
    ("sam_groceries",    "facebookresearch/segment-anything/main/notebooks/images/groceries.jpg",
"segment-anything", _APACHE, _NOT_DOC, "object"),
    ("sam_truck",        "facebookresearch/segment-anything/main/notebooks/images/truck.jpg",
"segment-anything", _APACHE, _NOT_DOC, "urban_architecture"),
    ("mmd_demo",         "open-mmlab/mmdetection/main/demo/demo.jpg", "mmdetection", _APACHE,
_NOT_DOC, "natural_scene"),
    ("hf_cats",
"huggingface/transformers/main/tests/fixtures/tests_samples/COCO/000000039769.png", "transformers
test fixtures", _APACHE, "COCO val image (Flickr, CC BY 2.0 per COCO terms; attribution per COCO)",
"animal"),
    ("thr_park",         THR + "cube/Park2/posx.jpg",                "three.js examples", "MIT",
"cube-map face; texture rights " + _NOT_DOC, "natural_scene"),
    ("thr_bridge",       THR + "cube/Bridge2/posx.jpg",              "three.js examples", "MIT",
"cube-map face; texture rights " + _NOT_DOC, "natural_scene"),
    ("thr_castle",       THR + "cube/SwedishRoyalCastle/px.jpg",     "three.js examples", "MIT",
"cube-map face (night scene); texture rights " + _NOT_DOC, "urban_architecture"),
    ("thr_pisa",         THR + "cube/pisa/px.png",                   "three.js examples", "MIT",
"cube-map face; texture rights " + _NOT_DOC, "urban_architecture"),
    ("thr_milkyway",     THR + "cube/MilkyWay/dark-s_px.jpg",        "three.js examples", "MIT",
"cube-map face (near-black sky); texture rights " + _NOT_DOC, "astronomy_space"),
    ("thr_caravaggio",   THR + "758px-Canestra_di_frutta_%28Caravaggio%29.jpg", "three.js examples",
"MIT", "reproduction of a public-domain painting (Caravaggio)", "painting_art"),
    ("thr_interior",     THR + "2294472375_24a3b8ef46_o.jpg",        "three.js examples", "MIT",
"equirectangular interior panorama (Flickr origin); " + _NOT_DOC, "indoor_object"),
    ("thr_earth",        THR + "planets/earth_atmos_2048.jpg",       "three.js examples", "MIT",
"NASA-derived Earth map; " + _NOT_DOC, "scientific_map"),
    ("thr_moonmap",      THR + "planets/moon_1024.jpg",              "three.js examples", "MIT",
"lunar surface map; " + _NOT_DOC, "scientific_map"),
    ("thr_hardwood",     THR + "hardwood2_diffuse.jpg",              "three.js examples", "MIT",
"wood-floor texture; " + _NOT_DOC, "texture_natural"),
    ("thr_brick",        THR + "brick_diffuse.jpg",                  "three.js examples", "MIT",
"brick-wall texture; " + _NOT_DOC, "texture_natural"),
    ("thr_grass",        THR + "terrain/grasslight-big.jpg",         "three.js examples", "MIT",
"grass texture; " + _NOT_DOC, "texture_natural"),
    ("thr_lava",         THR + "lava/lavatile.jpg",                  "three.js examples", "MIT",
"lava texture; " + _NOT_DOC, "texture_natural"),
    ("ski_astronaut",    SKI + "astronaut.png", "scikit-image data", "BSD-3-Clause", "NASA photograph
of Eileen Collins (public domain)", "person_portrait"),
    ("ski_brick",        SKI + "brick.png",     "scikit-image data", "BSD-3-Clause", "CC0
(CC0Textures)", "texture_natural"),
    ("ski_camera",       SKI + "camera.png",    "scikit-image data", "BSD-3-Clause", "CC0 by the
photographer (Lav Varshney)", "person_portrait"),
    ("ski_cell",         SKI + "cell.png",      "scikit-image data", "BSD-3-Clause", "quantitative
phase image of a cell (qpformat)", "scientific_medical"),
    ("ski_chelsea",      SKI + "chelsea.png",   "scikit-image data", "BSD-3-Clause", "CC0 by the
photographer (Stefan van der Walt)", "animal"),
    ("ski_clock",        SKI + "clock_motion.png", "scikit-image data", "BSD-3-Clause", "public
domain (Stefan van der Walt); motion-blurred", "object"),
```

```
    ("ski_coffee",       SKI + "coffee.png",    "scikit-image data", "BSD-3-Clause", "CC0 by the
photographer (Rachel Michetti)", "object"),
    ("ski_coins",        SKI + "coins.png",     "scikit-image data", "BSD-3-Clause", "Brooklyn Museum
Collection (open collection)", "object"),
    ("ski_grass",        SKI + "grass.png",     "scikit-image data", "BSD-3-Clause", "CC0
(DeviantArt, linolafett)", "texture_natural"),
    ("ski_gravel",       SKI + "gravel.png",    "scikit-image data", "BSD-3-Clause", "CC0
(CC0Textures)", "texture_natural"),
    ("ski_hubble",       SKI + "hubble_deep_field.jpg", "scikit-image data", "BSD-3-Clause",
"HubbleSite / NASA image", "astronomy_space"),
    ("ski_ihc",          SKI + "ihc.png",       "scikit-image data", "BSD-3-Clause",
"immunohistochemistry micrograph (Center for Microscopy and Image Analysis, Zurich)",
"scientific_medical"),
    ("ski_moon",         SKI + "moon.png",      "scikit-image data", "BSD-3-Clause", "low-contrast
lunar surface image", "astronomy_space"),
    ("ski_motorcycle",   SKI + "motorcycle_left.png", "scikit-image data", "BSD-3-Clause",
"Middlebury stereo dataset (left view only); research use", "indoor_object"),
    ("ski_retina",       SKI + "retina.jpg",    "scikit-image data", "BSD-3-Clause", "CC0 (Wikimedia
Commons fundus photograph)", "scientific_medical"),
    ("ski_rocket",       SKI + "rocket.jpg",    "scikit-image data", "BSD-3-Clause", "SpaceX Photos
(Flickr); see upstream licence", "aerial_vehicle"),
    ("skl_china",        "scikit-learn/scikit-learn/main/sklearn/datasets/images/china.jpg",
"scikit-learn", "BSD-3-Clause", "CC BY 2.0, danielbuechele (Flickr)", "urban_architecture"),
    ("skl_flower",       "scikit-learn/scikit-learn/main/sklearn/datasets/images/flower.jpg",
"scikit-learn", "BSD-3-Clause", "CC BY 2.0, vultilion (Flickr)", "plant_object"),
    ("mpl_hopper",
"matplotlib/matplotlib/main/lib/matplotlib/mpl-data/sample_data/grace_hopper.jpg", "matplotlib",
"PSF-based (matplotlib licence)", "official US Navy photograph (public domain)", "person_portrait"),
    ("scp_face",         "scipy:face",   "scipy.datasets", "BSD-3-Clause", "raccoon image derived
from pixnio.com", "animal"),
    ("scp_ascent",       "scipy:ascent", "scipy.datasets", "BSD-3-Clause", "derived from pixnio.com
('accent-to-the-top')", "person_sports"),
    ("iio_bricks",       "imageio/imageio-binaries/master/images/bricks.jpg", "imageio-binaries",
"BSD-2-Clause", _NOT_DOC, "texture_natural"),
    ("iio_wikkie",       "imageio/imageio-binaries/master/images/wikkie.png", "imageio-binaries",
"BSD-2-Clause", _NOT_DOC, "animal"),
    ("pil_plumeria",     PIL + "flower.jpg",      "Pillow tests", "HPND", _NOT_DOC, "plant_object"),
    ("pil_heather",      PIL + "flower2.jpg",     "Pillow tests", "HPND", _NOT_DOC, "plant_object"),
    ("pil_field",        PIL + "junk_jpeg_header.jpg", "Pillow tests", "HPND", _NOT_DOC,
"natural_scene"),
    ("pil_sign",         PIL + "iptc.jpg",        "Pillow tests", "HPND", _NOT_DOC,
"urban_architecture"),
    ("pil_cabin",        PIL + "sugarshack_bad_mpo_header.jpg", "Pillow tests", "HPND", _NOT_DOC,
"natural_scene"),
    ("pil_coins",        PIL + "xmp_test.jpg",    "Pillow tests", "HPND", _NOT_DOC, "object"),
    ("pil_aircraft",     PIL + "exif_gps_typeerror.jpg", "Pillow tests", "HPND", _NOT_DOC,
"aerial_vehicle"),
    ("pil_beach",        PIL + "icc_profile_big.jpg", "Pillow tests", "HPND", _NOT_DOC,
"natural_scene"),
    ("pil_mountain",     PIL + "exif-72dpi-int.jpg",  "Pillow tests", "HPND", _NOT_DOC,
"natural_scene"),
]

EXCLUDED = {
    "lena.jpg": "contested provenance/consent; many institutions advise against its use",
    "left02..left14 (OpenCV calibration frames)": "near-identical frames of one calibration scene",
    "aloeR / right.jpg / rubberwhale2 / motorcycle_right": "second view of an already-included stereo
pair or frame",
    "meadow_cube.jpg": "2048x12288 strip, not a natural image of a scene",
    "images < 240 px on the short side": "up-sampling to 512 px would make them artificially smooth",
    "Pillow format-edge-case images (hopper_* variants, EXIF/ICC tests)": "same picture in many
encodings; not independent content",
    "ultralytics bus.jpg / zidane.jpg": "AGPL-3.0 repository; photographs of identifiable persons
with unclear rights",
}
```

2.2 build_dataset.py

Downloads, standardises and records the 100 images. 209 lines.

```
from __future__ import annotations

import argparse
import csv
import hashlib
import json
import logging
import os
```

```
import sys
import time
import urllib.parse
import urllib.request

import numpy as np
from PIL import Image
from scipy import ndimage

from dataset_manifest import REAL_IMAGES
from image_io import ImageLoadError, load_gray_square, save_gray_png

LOG = logging.getLogger("build_dataset")
RAW = "https://raw.githubusercontent.com/"
SIZE = 512


def sha256_file(path: str) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1 << 20), b""):
            h.update(chunk)
    return h.hexdigest()


def dhash64(arr: np.ndarray) -> int:
    im = Image.fromarray(np.clip(arr, 0, 255).astype(np.uint8)).resize((9, 8), Image.LANCZOS)
    a = np.asarray(im, dtype=np.int16)
    bits = (a[:, 1:] > a[:, :-1]).flatten()
    return int("".join("1" if b else "0" for b in bits), 2)


def hamming(a: int, b: int) -> int:
    return bin(a ^ b).count("1")


def download(rel: str, dest: str, retries: int = 3) -> bool:
    url = RAW + rel
    last = None
    for k in range(retries):
        try:
            with urllib.request.urlopen(urllib.request.Request(url, headers={"User-Agent": "ivp-
watermark-build/1.0"}), timeout=60) as r:
                data = r.read()
            if not data:
                raise IOError("empty response")
            with open(dest, "wb") as f:
                f.write(data)
            return True
        except Exception as exc:
            last = exc
            time.sleep(1.5 * (k + 1))
    LOG.warning("download failed %s (%s)", url, last)
    return False


def fetch_scipy(name: str, dest: str) -> bool:
    try:
        import scipy.datasets as sd
        arr = getattr(sd, name)()
        Image.fromarray(arr).save(dest)
        return True
    except Exception as exc:
        LOG.warning("scipy.datasets.%s unavailable: %s", name, exc)
        return False


def _smooth_noise(rng, sigma, mean, std):
    n = ndimage.gaussian_filter(rng.standard_normal((SIZE, SIZE)), sigma)
    n = (n - n.mean()) / n.std()
    return np.clip(mean + std * n, 0, 255)


def synth_images():
    rng = np.random.default_rng(20260713)
    yy, xx = np.mgrid[0:SIZE, 0:SIZE].astype(np.float64)
    out = []
```

```
    out.append(("syn_hgradient", "horizontal linear gradient 0-255", np.tile(np.linspace(0, 255,
SIZE), (SIZE, 1))))
    r = np.hypot(xx - SIZE / 2, yy - SIZE / 2)
    out.append(("syn_radial", "radial gradient (smooth, low detail)", np.clip(255 - r * 0.7, 0,
255)))
    out.append(("syn_grating", "2-D sinusoidal grating (periodic texture)", 128 + 100 * np.sin(2 *
np.pi * (xx * 9 + yy * 5) / SIZE)))
    out.append(("syn_white_noise", "white Gaussian noise, mean 128 / std 40", np.clip(128 + 40 *
rng.standard_normal((SIZE, SIZE)), 0, 255)))
    out.append(("syn_lowcontrast", "smooth noise, mean 128 / std 8 (very low contrast)",
_smooth_noise(rng, 6, 128, 8)))
    out.append(("syn_checker", "32-px checkerboard (hard edges)", 255.0 * (((xx // 32) + (yy // 32))
% 2)))
    out.append(("syn_dark", "smooth noise, mean 25 / std 12 (dark)", _smooth_noise(rng, 5, 25, 12)))
    out.append(("syn_bright", "smooth noise, mean 230 / std 12 (bright)", _smooth_noise(rng, 5, 230,
12)))
    f = np.fft.fftfreq(SIZE)
    fx, fy = np.meshgrid(f, f)
    amp = 1.0 / np.maximum(np.hypot(fx, fy), 1.0 / SIZE)
    spec = amp * np.exp(2j * np.pi * rng.random((SIZE, SIZE)))
    frac = np.real(np.fft.ifft2(spec))
    frac = (frac - frac.mean()) / frac.std()
    out.append(("syn_fractal", "1/f fractal noise texture", np.clip(128 + 45 * frac, 0, 255)))
    scene = np.full((SIZE, SIZE), 110.0)
    for _ in range(14):
        x0, y0 = rng.integers(0, SIZE - 80, 2)
        w, h = rng.integers(40, 160, 2)
        scene[y0:y0 + h, x0:x0 + w] = rng.integers(20, 240)
    out.append(("syn_blocks", "random grey rectangles on grey background", scene))
    return out


def main():
    ap = argparse.ArgumentParser()
    here = os.path.dirname(os.path.abspath(__file__))
    ap.add_argument("--out", default=os.path.join(here, "..", "..", "data"))
    ap.add_argument("--target", type=int, default=100)
    ap.add_argument("--dup-threshold", type=int, default=6)
    args = ap.parse_args()
    out = os.path.abspath(args.out)
    src_dir = os.path.join(out, "source_images")
    orig_dir = os.path.join(out, "original")
    os.makedirs(src_dir, exist_ok=True)
    os.makedirs(orig_dir, exist_ok=True)
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")

    rows, hashes, skipped = [], [], []
    for key, rel, group, lic, note, category in REAL_IMAGES:
        ext = os.path.splitext(urllib.parse.unquote(rel))[1] or ".png"
        if rel.startswith("scipy:"):
            dest = os.path.join(src_dir, f"{key}.png")
            ok = os.path.isfile(dest) or fetch_scipy(rel.split(":", 1)[1], dest)
            url = f"scipy.datasets.{rel.split(':', 1)[1]}()"
        else:
            dest = os.path.join(src_dir, f"{key}{ext}")
            ok = os.path.isfile(dest) and os.path.getsize(dest) > 0 or download(rel, dest)
            url = RAW + rel
        if not ok:
            skipped.append((key, "download failed"))
            continue
        try:
            arr, info = load_gray_square(dest, SIZE)
        except ImageLoadError as exc:
            skipped.append((key, str(exc)))
            LOG.warning("skip %s: %s", key, exc)
            continue
        if min(info.width, info.height) < 240:
            skipped.append((key, f"too small ({info.width}x{info.height})"))
            continue
        hsh = dhash64(arr)
        dup = next((k for k, h in hashes if hamming(h, hsh) <= args.dup_threshold), None)
        if dup:
            skipped.append((key, f"near-duplicate of {dup}"))
            LOG.info("duplicate %s ~ %s", key, dup)
            continue
        hashes.append((key, hsh))
        rows.append(dict(key=key, arr=arr, url=url, group=group, lic=lic, note=note,
                         category=category, info=info, sha=sha256_file(dest),
```

```
                        fname=os.path.basename(dest), synthetic=False))

    need = args.target - len(rows)
    if need < 0:
        rows = rows[:args.target]
        need = 0
    synth = synth_images()
    if need > len(synth):
        LOG.warning("only %d synthetic generators available; dataset will have %d images",
                    len(synth), len(rows) + len(synth))
    for key, desc, arr in synth[:need]:
        rows.append(dict(key=key, arr=arr, url="generated by build_dataset.py (seeded)",
group="synthetic control",
                        lic="n/a (generated)", note=desc, category="synthetic_control",
                        info=None, sha="", fname="", synthetic=True))

    meta_path = os.path.join(out, "metadata.csv")
    cols = ["image_id", "key", "original_filename", "source_group", "source_url",
"repository_license",
            "rights_note", "original_width", "original_height", "original_mode", "original_format",
            "original_file_size_bytes", "source_sha256", "standardized_shape",
"standardized_channels",
            "category", "is_synthetic", "mean_intensity", "contrast_std", "brightness_class",
            "contrast_class", "applicable_operations"]
    with open(meta_path, "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=cols)
        w.writeheader()
        for i, r in enumerate(rows, 1):
            iid = f"image_{i:03d}"
            save_gray_png(r["arr"], os.path.join(orig_dir, iid + ".png"))
            a = np.rint(r["arr"])
            mean, std = float(a.mean()), float(a.std())
            info = r["info"]
            w.writerow(dict(
                image_id=iid, key=r["key"], original_filename=r["fname"], source_group=r["group"],
                source_url=r["url"], repository_license=r["lic"], rights_note=r["note"],
                original_width=info.width if info else SIZE, original_height=info.height if info else
SIZE,
                original_mode=info.mode if info else "generated", original_format=info.fmt if info
else "PNG",
                original_file_size_bytes=info.file_size if info else "", source_sha256=r["sha"],
                standardized_shape=f"{SIZE}x{SIZE}", standardized_channels=1, category=r["category"],
                is_synthetic=int(r["synthetic"]), mean_intensity=round(mean, 2),
contrast_std=round(std, 2),
                brightness_class="dark" if mean < 80 else ("bright" if mean > 170 else "mid"),
                contrast_class="low" if std < 35 else ("high" if std > 65 else "medium"),
                applicable_operations="embed;extract;14 attacks + reference;PSNR;SSIM;NC;BER;alpha-
sweep;controls"))

    with open(os.path.join(out, "build_log.json"), "w") as f:
        json.dump({"images_written": len(rows), "real": sum(not r["synthetic"] for r in rows),
                   "synthetic": sum(r["synthetic"] for r in rows), "skipped": skipped}, f, indent=2)
    LOG.info("wrote %d images (%d real, %d synthetic); skipped %d",
             len(rows), sum(not r["synthetic"] for r in rows), sum(r["synthetic"] for r in rows),
len(skipped))
    for k, why in skipped:
        LOG.info("  skipped %-16s %s", k, why)
    return 0 if len(rows) == args.target else 1


if __name__ == "__main__":
    sys.exit(main())
```

Part 3: Experiments

Scripts that run the experiments and record results.

3.1 run_experiment.py

Original two image experiment. 152 lines.

```
import os
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from PIL import Image
from skimage import data as skdata
from skimage.transform import resize

from watermark_core import embed_watermark, extract_watermark
from metrics import psnr, ssim, normalized_correlation, bit_error_rate
from attacks import ATTACK_SUITE
from logo import make_watermark_logo

RESULTS_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "..", "results",
"legacy_two_image")
os.makedirs(RESULTS_DIR, exist_ok=True)


def load_host_images():
    hosts = {
        "cameraman": resize(skdata.camera().astype(np.float64), (512, 512), preserve_range=True),
        "astronaut_gray": resize(skdata.astronaut().mean(axis=2).astype(np.float64), (512, 512),
preserve_range=True),
    }
    return hosts


def save_image(arr, path):
    Image.fromarray(np.clip(arr, 0, 255).astype(np.uint8)).save(path)


def main():
    alpha = 0.10
    hosts = load_host_images()
    watermark = make_watermark_logo(size=64, text="CYBERION")
    save_image(watermark, os.path.join(RESULTS_DIR, "watermark_logo.png"))

    all_rows = []
    per_host_extracted = {}

    for host_name, host in hosts.items():
        watermarked, key = embed_watermark(host, watermark, alpha=alpha)
        save_image(host, os.path.join(RESULTS_DIR, f"{host_name}_original.png"))
        save_image(watermarked, os.path.join(RESULTS_DIR, f"{host_name}_watermarked.png"))

        imperceptibility_psnr = psnr(host, watermarked)
        imperceptibility_ssim = ssim(host, watermarked)

        extracted_examples = {}
        for attack_name, attack_fn in ATTACK_SUITE.items():
            attacked = attack_fn(watermarked)
            extracted = extract_watermark(attacked, key)
            nc = normalized_correlation(watermark, extracted)
            ber = bit_error_rate(watermark, extracted)

            all_rows.append({
                "Host Image": host_name,
                "Attack": attack_name,
                "Watermarked PSNR (dB)": round(imperceptibility_psnr, 3),
                "Watermarked SSIM": round(imperceptibility_ssim, 4),
                "Extracted NC": round(nc, 4),
                "Extracted BER": round(ber, 4),
            })
            extracted_examples[attack_name] = extracted
```

```
        per_host_extracted[host_name] = extracted_examples

    df = pd.DataFrame(all_rows)
    df.to_csv(os.path.join(RESULTS_DIR, "results_table.csv"), index=False)
    print(df.to_string(index=False))

    for host_name, host in hosts.items():
        watermarked = np.array(Image.open(os.path.join(RESULTS_DIR,
f"{host_name}_watermarked.png"))).astype(np.float64)
        diff = np.abs(host - watermarked)
        fig, axes = plt.subplots(1, 3, figsize=(12, 4.2))
        axes[0].imshow(host, cmap="gray"); axes[0].set_title("Original Host"); axes[0].axis("off")
        axes[1].imshow(watermarked, cmap="gray"); axes[1].set_title("Watermarked");
axes[1].axis("off")
        im = axes[2].imshow(diff, cmap="hot"); axes[2].set_title("Absolute Difference (x1)");
axes[2].axis("off")
        fig.colorbar(im, ax=axes[2], fraction=0.046)
        plt.tight_layout()
        plt.savefig(os.path.join(RESULTS_DIR, f"{host_name}_comparison.png"), dpi=150)
        plt.close(fig)

    gallery_attacks = ["No Attack", "JPEG Q=50", "JPEG Q=20", "Gaussian Noise (var=0.02)",
                       "Median Filter 3x3", "Rotation 15 deg", "Cropping 25%", "Scaling 0.5x"]
    fig, axes = plt.subplots(2, 4, figsize=(14, 7))
    for ax, aname in zip(axes.flat, gallery_attacks):
        ex = per_host_extracted["cameraman"][aname]
        ax.imshow(ex, cmap="gray")
        row = df[(df["Host Image"] == "cameraman") & (df["Attack"] == aname)].iloc[0]
        ax.set_title(f"{aname}\nNC={row['Extracted NC']:.3f}", fontsize=9)
        ax.axis("off")
    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "extraction_gallery.png"), dpi=150)
    plt.close(fig)

    sub = df[df["Host Image"] == "cameraman"].sort_values("Extracted NC")
    fig, ax = plt.subplots(figsize=(9, 6))
    colors = ["#c0392b" if v < 0.75 else "#27ae60" for v in sub["Extracted NC"]]
    ax.barh(sub["Attack"], sub["Extracted NC"], color=colors)
    ax.axvline(0.75, color="black", linestyle="--", linewidth=1, label="Recognizability threshold
(NC=0.75)")
    ax.set_xlabel("Normalized Correlation (NC)")
    ax.set_title("Watermark Robustness Across Attack Types (host: cameraman)")
    ax.legend(loc="lower right")
    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "nc_bar_chart.png"), dpi=150)
    plt.close(fig)

    host = hosts["cameraman"]
    alphas = [0.02, 0.05, 0.08, 0.10, 0.15, 0.20, 0.30, 0.40]
    sweep_rows = []
    for a in alphas:
        wm_img, k = embed_watermark(host, watermark, alpha=a)
        p = psnr(host, wm_img)
        attacked = ATTACK_SUITE["JPEG Q=50"](wm_img)
        ext = extract_watermark(attacked, k)
        nc = normalized_correlation(watermark, ext)
        sweep_rows.append({"alpha": a, "PSNR": p, "NC_after_JPEG50": nc})
    sweep_df = pd.DataFrame(sweep_rows)
    sweep_df.to_csv(os.path.join(RESULTS_DIR, "alpha_sweep.csv"), index=False)

    fig, ax1 = plt.subplots(figsize=(8, 5.5))
    ax2 = ax1.twinx()
    ax1.plot(sweep_df["alpha"], sweep_df["PSNR"], "o-", color="#2980b9", label="PSNR
(imperceptibility)")
    ax2.plot(sweep_df["alpha"], sweep_df["NC_after_JPEG50"], "s-", color="#e67e22", label="NC after
JPEG Q=50 (robustness)")
    ax1.set_xlabel("Embedding strength (alpha)")
    ax1.set_ylabel("PSNR (dB)", color="#2980b9")
    ax2.set_ylabel("NC after JPEG-50 attack", color="#e67e22")
    ax1.set_title("Imperceptibility vs. Robustness Trade-off")
    fig.legend(loc="upper center", bbox_to_anchor=(0.5, 0.02), ncol=2)
    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "alpha_tradeoff.png"), dpi=150, bbox_inches="tight")
    plt.close(fig)

    summary = {
        "alpha_used_main_experiment": alpha,
        "hosts": list(hosts.keys()),
```

```
        "num_attacks_tested": len(ATTACK_SUITE),
        "mean_watermarked_psnr_db": round(df["Watermarked PSNR (dB)"].mean(), 3),
        "mean_watermarked_ssim": round(df["Watermarked SSIM"].mean(), 4),
        "mean_nc_all_attacks": round(df["Extracted NC"].mean(), 4),
        "attacks_with_NC_above_0.75": int((df["Extracted NC"] > 0.75).sum()),
        "attacks_total_rows": len(df),
    }
    with open(os.path.join(RESULTS_DIR, "summary.json"), "w") as f:
        json.dump(summary, f, indent=2)
    print("\nSUMMARY:", json.dumps(summary, indent=2))


if __name__ == "__main__":
    main()
```

3.2 run_batch_experiment.py

Batch experiment on 100 images. 258 lines.

```
from __future__ import annotations

import os

for _v in ("OPENBLAS_NUM_THREADS", "OMP_NUM_THREADS", "MKL_NUM_THREADS"):
    os.environ.setdefault(_v, "1")

import argparse
import concurrent.futures as cf
import json
import logging
import platform
import resource
import sys
import time

import numpy as np
import pandas as pd

from attacks import ATTACK_FAMILY, ATTACK_NAMES, make_attack_suite
from image_io import save_gray_png, verify_png
from logo import make_watermark_logo
from metrics import bit_error_rate, normalized_correlation, psnr, ssim, zero_normalized_correlation
from watermark_core import embed_watermark, extract_watermark

HERE = os.path.dirname(os.path.abspath(__file__))
LOG = logging.getLogger("batch")

DEFAULT_CONFIG = {
    "alpha": 0.10,
    "image_size": 512,
    "logo_size": 64,
    "logo_text": "CYBERION",
    "other_logos": ["OTHERLOGO", "OWNER-B"],
    "protocols": ["uint8", "float"],
    "primary_protocol": "uint8",
    "base_seed": 20260713,
    "alphas": [0.02, 0.05, 0.08, 0.10, 0.15, 0.20, 0.30, 0.40],
    "sweep_attacks": ["JPEG Q=50", "Gaussian Noise (var=0.02)", "Gaussian Blur (sigma=1.5)",
"Rotation 5 deg"],
    "n_controls_per_image": 5,
    "nc_threshold": 0.75,
    "data_dir": "../../data",
    "results_dir": "../../results",
    "outputs_dir": "../../outputs",
}


def _t():
    return time.perf_counter()


def _load_host(data_dir, image_id):
    from PIL import Image
    a = np.asarray(Image.open(os.path.join(data_dir, "original", image_id + ".png")),
dtype=np.float64)
    if a.ndim != 2:
        raise ValueError(f"{image_id}: expected greyscale, got shape {a.shape}")
    return a
```

```
def _quantize(a):
    return np.clip(np.rint(a), 0, 255)


def process_image(task):
    idx, image_id, cfg, others = task
    data_dir = cfg["data_dir"]
    rows_att, rows_ctl, rows_sw = [], [], []
    rec = {"image_id": image_id, "status": "ok", "error": ""}
    try:
        host = _load_host(data_dir, image_id)
        logo = make_watermark_logo(cfg["logo_size"], cfg["logo_text"])
        alpha = cfg["alpha"]

        t0 = _t()
        wm_float, key = embed_watermark(host, logo, alpha=alpha)
        rec["t_embed_ms"] = (_t() - t0) * 1e3
        wm_u8 = _quantize(wm_float)
        out_path = os.path.join(cfg["outputs_dir"], "watermarked", f"{image_id}_wm.png")
        save_gray_png(wm_u8, out_path)
        rec["output_valid"] = bool(verify_png(out_path, host.shape))
        rec["psnr_float_db"] = psnr(host, wm_float)
        rec["ssim_float"] = ssim(host, wm_float)
        rec["psnr_db"] = psnr(host, wm_u8)
        rec["ssim"] = ssim(host, wm_u8)
        rec["max_abs_diff"] = float(np.abs(host - wm_u8).max())
        rec["mean_abs_diff"] = float(np.abs(host - wm_u8).mean())
        rec["key_bytes"] = int(sum(v.nbytes for v in key.values() if isinstance(v, np.ndarray)))

        suite = make_attack_suite(cfg["base_seed"] + 100 * idx)
        for protocol in cfg["protocols"]:
            base = wm_u8 if protocol == "uint8" else wm_float
            for name in ATTACK_NAMES:
                row = {"image_id": image_id, "protocol": protocol, "attack": name,
                       "family": ATTACK_FAMILY[name], "status": "ok", "error": ""}
                try:
                    t0 = _t()
                    attacked = suite[name](base)
                    row["t_attack_ms"] = (_t() - t0) * 1e3
                    t0 = _t()
                    ext = extract_watermark(attacked, key)
                    row["t_extract_ms"] = (_t() - t0) * 1e3
                    row["nc"] = normalized_correlation(logo, ext)
                    row["zncc"] = zero_normalized_correlation(logo, ext)
                    row["ber"] = bit_error_rate(logo, ext)
                except Exception as exc:
                    row.update(status="error", error=f"{type(exc).__name__}: {exc}")
                rows_att.append(row)

        sweep_suite = make_attack_suite(cfg["base_seed"] + 100 * idx + 50)
        for a in cfg["alphas"]:
            try:
                w, k = embed_watermark(host, logo, alpha=a)
                wq = _quantize(w)
                p_a, s_a = psnr(host, wq), ssim(host, wq)
            except Exception as exc:
                rows_sw.append({"image_id": image_id, "alpha": a, "attack": "(embed)", "status":
"error",
                                "error": f"{type(exc).__name__}: {exc}"})
                continue
            for an in ["No Attack"] + list(cfg["sweep_attacks"]):
                row = {"image_id": image_id, "alpha": a, "attack": an, "psnr_db": p_a, "ssim": s_a,
                       "status": "ok", "error": ""}
                try:
                    ext = extract_watermark(sweep_suite[an](wq), k)
                    row["nc"] = normalized_correlation(logo, ext)
                    row["zncc"] = zero_normalized_correlation(logo, ext)
                    row["ber"] = bit_error_rate(logo, ext)
                except Exception as exc:
                    row.update(status="error", error=f"{type(exc).__name__}: {exc}")
                rows_sw.append(row)

        def ctl(kind, other, ncv, berv, zz, extra=""):
            rows_ctl.append({"image_id": image_id, "control": kind, "other": other,
                             "nc": ncv, "ber": berv, "zncc": zz, "note": extra})
```

```
        ext = extract_watermark(wm_u8, key)
        ctl("genuine (own key, no attack)", image_id, normalized_correlation(logo, ext),
bit_error_rate(logo, ext), zero_normalized_correlation(logo, ext))
        ext = extract_watermark(host, key)
        ctl("d: own host unwatermarked, own key", image_id, normalized_correlation(logo, ext),
bit_error_rate(logo, ext), zero_normalized_correlation(logo, ext),
            extra="degenerate: S_att == S_original so Sw' = 0")
        for oid in others:
            oh = _load_host(data_dir, oid)
            ext = extract_watermark(oh, key)
            ctl("a: other host unwatermarked, key_i", oid, normalized_correlation(logo, ext),
bit_error_rate(logo, ext), zero_normalized_correlation(logo, ext))
            ow, _ = embed_watermark(oh, logo, alpha=alpha)
            ext = extract_watermark(_quantize(ow), key)
            ctl("b: other host watermarked, key_i", oid, normalized_correlation(logo, ext),
bit_error_rate(logo, ext), zero_normalized_correlation(logo, ext))
        for text in cfg["other_logos"]:
            logo_b = make_watermark_logo(cfg["logo_size"], text)
            wb, kb = embed_watermark(host, logo_b, alpha=alpha)
            wbq = _quantize(wb)
            ext_a = extract_watermark(wbq, key)
            ext_b = extract_watermark(wbq, kb)
            ctl("c: same host, other logo B, decoded with key A (NC vs A)", text,
                normalized_correlation(logo, ext_a), bit_error_rate(logo, ext_a),
zero_normalized_correlation(logo, ext_a))
            ctl("c-ref: same host, logo B, decoded with key B (NC vs B)", text,
                normalized_correlation(logo_b, ext_b), bit_error_rate(logo_b, ext_b),
zero_normalized_correlation(logo_b, ext_b))
    except Exception as exc:
        rec.update(status="error", error=f"{type(exc).__name__}: {exc}")
        LOG.exception("image %s failed", image_id)
    rec["peak_rss_mb"] = resource.getrusage(resource.RUSAGE_SELF).ru_maxrss / 1024.0
    return rec, rows_att, rows_sw, rows_ctl


def env_info():
    import PIL, pywt, scipy, skimage
    cpu = ""
    try:
        with open("/proc/cpuinfo") as f:
            cpu = next(l.split(":", 1)[1].strip() for l in f if l.startswith("model name"))
    except Exception:
        pass
    return {"python": sys.version.split()[0], "platform": platform.platform(), "cpu": cpu,
            "cpu_count": os.cpu_count(), "numpy": np.__version__, "scipy": scipy.__version__,
            "scikit-image": skimage.__version__, "PyWavelets": pywt.__version__, "Pillow":
PIL.__version__,
            "pandas": pd.__version__}


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--config", default=None)
    ap.add_argument("--workers", type=int, default=os.cpu_count() or 1)
    ap.add_argument("--limit", type=int, default=None, help="process only the first N images
(debug)")
    args = ap.parse_args()
    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")

    cfg = dict(DEFAULT_CONFIG)
    if args.config:
        with open(args.config) as f:
            cfg.update(json.load(f))
    for k in ("data_dir", "results_dir", "outputs_dir"):
        cfg[k] = os.path.abspath(os.path.join(HERE, cfg[k]))
    os.makedirs(cfg["results_dir"], exist_ok=True)
    os.makedirs(os.path.join(cfg["outputs_dir"], "watermarked"), exist_ok=True)

    meta = pd.read_csv(os.path.join(cfg["data_dir"], "metadata.csv"))
    if args.limit:
        meta = meta.head(args.limit)
    ids = list(meta["image_id"])
    n = len(ids)
    rng = np.random.default_rng(cfg["base_seed"])
    tasks = []
    for i, iid in enumerate(ids):
        pool = [x for x in ids if x != iid]
```

```
        others = list(rng.choice(pool, size=min(cfg["n_controls_per_image"], len(pool)),
replace=False))
        tasks.append((i, iid, cfg, others))

    LOG.info("processing %d images with %d worker(s)", n, args.workers)
    t_start = _t()
    results = []
    with cf.ProcessPoolExecutor(max_workers=args.workers) as ex:
        for res in ex.map(process_image, tasks):
            results.append(res)
            if len(results) % 10 == 0:
                LOG.info("  %d/%d images done", len(results), n)
    wall = _t() - t_start

    img_df = pd.DataFrame([r[0] for r in results]).sort_values("image_id")
    att_df = pd.DataFrame([row for r in results for row in r[1]])
    sw_df = pd.DataFrame([row for r in results for row in r[2]])
    ctl_df = pd.DataFrame([row for r in results for row in r[3]])
    rd = cfg["results_dir"]
    img_df.to_csv(os.path.join(rd, "results_images.csv"), index=False)
    att_df.to_csv(os.path.join(rd, "results_attacks.csv"), index=False)
    sw_df.to_csv(os.path.join(rd, "results_alpha_sweep.csv"), index=False)
    ctl_df.to_csv(os.path.join(rd, "results_controls.csv"), index=False)

    full = pd.read_csv(os.path.join(cfg["data_dir"], "metadata.csv"))
    st = img_df.set_index("image_id")["status"]
    att_ok = att_df.groupby("image_id")["status"].apply(lambda s: int((s == "ok").sum()))
    full["processing_status"] = full["image_id"].map(st).fillna(full.get("processing_status", "not
processed"))
    full["attack_trials_ok"] = full["image_id"].map(att_ok).fillna(full.get("attack_trials_ok"))
    full["attack_trials_expected"] = len(ATTACK_NAMES) * len(cfg["protocols"])
    full.to_csv(os.path.join(cfg["data_dir"], "metadata.csv"), index=False)

    summary = {
        "config": {k: v for k, v in cfg.items() if not k.endswith("_dir")},
        "environment": env_info(),
        "images_requested": n,
        "images_ok": int((img_df["status"] == "ok").sum()),
        "watermarked_files_valid": int(img_df["output_valid"].fillna(False).sum()),
        "attack_trials_total": int(len(att_df)),
        "attack_trials_ok": int((att_df["status"] == "ok").sum()),
        "attack_trials_failed": int((att_df["status"] != "ok").sum()),
        "alpha_sweep_rows_ok": int((sw_df["status"] == "ok").sum()),
        "alpha_sweep_rows_total": int(len(sw_df)),
        "control_rows": int(len(ctl_df)),
        "wall_time_s": round(wall, 1),
        "workers": args.workers,
        "peak_rss_mb_max_over_workers": round(float(img_df["peak_rss_mb"].max()), 1),
    }
    with open(os.path.join(rd, "run_summary.json"), "w") as f:
        json.dump(summary, f, indent=2)
    LOG.info("summary: %s", json.dumps({k: v for k, v in summary.items() if k not in ("config",
"environment")}))
    return 0 if summary["attack_trials_failed"] == 0 and summary["images_ok"] == n else 1


if __name__ == "__main__":
    sys.exit(main())
```

3.3 video_demo.py

Per frame video test. 149 lines.

```
from __future__ import annotations

import os
for _v in ("OPENBLAS_NUM_THREADS", "OMP_NUM_THREADS", "MKL_NUM_THREADS"):
    os.environ.setdefault(_v, "1")

import argparse
import json
import subprocess
import tempfile
import time
import urllib.request

import cv2
import numpy as np
```

```
import pandas as pd
from PIL import Image

from logo import make_watermark_logo
from metrics import bit_error_rate, normalized_correlation, psnr
from watermark_core import embed_watermark, extract_watermark

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
RES = os.path.join(ROOT, "results")
CACHE = os.path.join(ROOT, "data", "source_videos")
CLIPS = {
    "vtest.avi": "opencv/opencv/4.x/samples/data/vtest.avi",
    "Megamind.avi": "opencv/opencv/4.x/samples/data/Megamind.avi",
}
SIZE = 512
CODECS = {
    "lossless (FFV1)":  ["-c:v", "ffv1"],
    "H.264 CRF 18":     ["-c:v", "libx264", "-crf", "18", "-pix_fmt", "yuv420p"],
    "H.264 CRF 23":     ["-c:v", "libx264", "-crf", "23", "-pix_fmt", "yuv420p"],
    "H.264 CRF 28":     ["-c:v", "libx264", "-crf", "28", "-pix_fmt", "yuv420p"],
    "MJPEG q=5":        ["-c:v", "mjpeg", "-q:v", "5", "-pix_fmt", "yuvj420p"],
    "MJPEG q=15":       ["-c:v", "mjpeg", "-q:v", "15", "-pix_fmt", "yuvj420p"],
}


def fetch(name):
    os.makedirs(CACHE, exist_ok=True)
    p = os.path.join(CACHE, name)
    if not os.path.exists(p):
        urllib.request.urlretrieve("https://raw.githubusercontent.com/" + CLIPS[name], p)
    return p


def gray_square(frame_bgr):
    g = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2GRAY)
    s = min(g.shape)
    y0, x0 = (g.shape[0] - s) // 2, (g.shape[1] - s) // 2
    g = g[y0:y0 + s, x0:x0 + s]
    return np.asarray(Image.fromarray(g).resize((SIZE, SIZE), Image.LANCZOS), dtype=np.float64)


def read_frames(path, n):
    cap = cv2.VideoCapture(path)
    if not cap.isOpened():
        raise IOError(f"cannot open {path}")
    fps = cap.get(cv2.CAP_PROP_FPS)
    total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    w, h = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)), int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    frames = []
    try:
        while len(frames) < n:
            ok, f = cap.read()
            if not ok:
                break
            frames.append(gray_square(f))
    finally:
        cap.release()
    return frames, dict(fps=fps, total_frames=total, width=w, height=h)


def encode_decode(frames, codec_args, fps, tmp):
    out = os.path.join(tmp, "clip." + ("mkv" if "ffv1" in codec_args else ("avi" if "mjpeg" in
codec_args else "mp4")))
    raw = b"".join(np.clip(np.rint(f), 0, 255).astype(np.uint8).tobytes() for f in frames)
    enc = subprocess.run(["ffmpeg", "-v", "error", "-y", "-f", "rawvideo", "-pix_fmt", "gray", "-s",
f"{SIZE}x{SIZE}",
                          "-r", str(fps), "-i", "-", *codec_args, out], input=raw,
capture_output=True)
    if enc.returncode != 0:
        raise RuntimeError(enc.stderr.decode()[:300])
    dec = subprocess.run(["ffmpeg", "-v", "error", "-i", out, "-f", "rawvideo", "-pix_fmt", "gray",
"-"], capture_output=True)
    if dec.returncode != 0:
        raise RuntimeError(dec.stderr.decode()[:300])
    buf = np.frombuffer(dec.stdout, dtype=np.uint8)
    n = buf.size // (SIZE * SIZE)
```

```
    decoded = [buf[i * SIZE * SIZE:(i + 1) * SIZE * SIZE].reshape(SIZE, SIZE).astype(np.float64) for
i in range(n)]
    return decoded, os.path.getsize(out)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--frames", type=int, default=60)
    args = ap.parse_args()
    logo = make_watermark_logo()
    rows, meta = [], {}
    for clip in CLIPS:
        frames, info = read_frames(fetch(clip), args.frames)
        meta[clip] = dict(info, frames_read=len(frames))
        t0 = time.perf_counter()
        marked, keys = [], []
        for f in frames:
            w, k = embed_watermark(f, logo, alpha=0.10)
            k = {kk: vv for kk, vv in k.items() if kk not in ("U", "Vt")}
            marked.append(np.clip(np.rint(w), 0, 255)); keys.append(k)
        t_embed = (time.perf_counter() - t0) / len(frames)
        fps = info["fps"] if info["fps"] and info["fps"] > 0 else 25
        with tempfile.TemporaryDirectory() as tmp:
            for codec, args_ in CODECS.items():
                row = dict(clip=clip, codec=codec, frames_in=len(frames), status="ok")
                try:
                    dec_wm, size_wm = encode_decode(marked, args_, fps, tmp)
                    dec_ref, size_ref = encode_decode(frames, args_, fps, tmp)
                    n = min(len(dec_wm), len(frames))
                    row["frames_decoded"] = len(dec_wm)
                    t0 = time.perf_counter()
                    ncs, bers = [], []
                    for i in range(n):
                        ex = extract_watermark(dec_wm[i], keys[i])
                        ncs.append(normalized_correlation(logo, ex));
bers.append(bit_error_rate(logo, ex))
                    row["t_extract_ms_per_frame"] = (time.perf_counter() - t0) / n * 1e3
                    row["psnr_wm_vs_orig_db"] = float(np.mean([psnr(frames[i], dec_wm[i]) for i in
range(n)]))
                    row["psnr_codec_only_db"] = float(np.mean([psnr(frames[i], dec_ref[i]) for i in
range(n)]))
                    row["nc_mean"] = float(np.mean(ncs)); row["nc_min"] = float(np.min(ncs))
                    row["frames_nc_ge_0_75"] = int(np.sum(np.array(ncs) >= 0.75)); row["ber_mean"] =
float(np.mean(bers))
                    row["size_wm_kb"] = size_wm / 1024; row["size_ref_kb"] = size_ref / 1024
                    if codec == "lossless (FFV1)":
                        keep = [i for i in range(len(dec_wm)) if i % 10 != 9]
                        naive = [normalized_correlation(logo, extract_watermark(dec_wm[j], keys[i]))
for i, j in enumerate(keep)]
                        aware = [normalized_correlation(logo, extract_watermark(dec_wm[j], keys[j]))
for j in keep]
                        row["framedrop_nc_naive_index"] = float(np.mean(naive))
                        row["framedrop_nc_with_frame_ids"] = float(np.mean(aware))
                except Exception as exc:
                    row.update(status=f"error: {type(exc).__name__}: {exc}")
                rows.append(row)
        meta[clip]["embed_ms_per_frame"] = t_embed * 1e3
        meta[clip]["embed_fps_single_thread"] = 1 / t_embed
        print(clip, meta[clip], flush=True)
    df = pd.DataFrame(rows)
    df.to_csv(os.path.join(RES, "video_demo_results.csv"), index=False)
    json.dump(meta, open(os.path.join(RES, "video_demo_meta.json"), "w"), indent=2)
    pd.set_option("display.width", 250)
    print(df.round(3).to_string())


if __name__ == "__main__":
    main()
```

3.4 exp_jpeg_truncation.py

Truncation versus rounding before JPEG. 28 lines.

```
import io

import numpy as np
from PIL import Image
from skimage import data as skdata
```

```
from skimage.transform import resize

from logo import make_watermark_logo
from metrics import normalized_correlation as nc, psnr
from watermark_core import embed_watermark, extract_watermark


def jpeg(img, q, mode):
    a = np.clip(img, 0, 255)
    a = a.astype(np.uint8) if mode == "truncate" else np.rint(a).astype(np.uint8)
    b = io.BytesIO(); Image.fromarray(a).save(b, "JPEG", quality=q); b.seek(0)
    return np.array(Image.open(b).convert("L")).astype(np.float64)


if __name__ == "__main__":
    wm = make_watermark_logo()
    host = resize(skdata.camera().astype(np.float64), (512, 512), preserve_range=True)
    print(f"{'alpha':>6} {'PSNR':>7} {'NC trunc.':>10} {'NC round':>9} {'mean(trunc.-float)':>19}")
    for a in (0.02, 0.05, 0.08, 0.10, 0.15, 0.20, 0.30, 0.40):
        w, k = embed_watermark(host, wm, alpha=a)
        t, r = jpeg(w, 50, "truncate"), jpeg(w, 50, "round")
        print(f"{a:6.2f} {psnr(host, w):7.2f} {nc(wm, extract_watermark(t, k)):10.3f} {nc(wm,
extract_watermark(r, k)):9.3f} "
              f"{float((jpeg(w, 100, 'truncate') - w).mean()):19.3f}")
```

3.5 exp_principal_component_variant.py

Principal component embedding variant. 44 lines.

```
import os
import numpy as np
import pywt
from PIL import Image
from skimage.transform import resize

from logo import make_watermark_logo
from metrics import normalized_correlation as nc, psnr
from watermark_core import dct2, idct2

ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", ".."))


def emb_pc(host, wm, alpha):
    LL, (LH, HL, HH) = pywt.dwt2(host, "haar")
    wr = resize(wm, LL.shape, anti_aliasing=True, preserve_range=True) / 255.0
    U, S, Vt = np.linalg.svd(dct2(LL))
    Uw, Sw, Vtw = np.linalg.svd(wr)
    D = np.diag(S) + alpha * (Uw * Sw)
    U1, S1, V1t = np.linalg.svd(D)
    out = pywt.idwt2((idct2((U * S1) @ Vt), (LH, HL, HH)), "haar")[:host.shape[0], :host.shape[1]]
    return np.clip(out, 0, 255), dict(U1=U1, V1t=V1t, S=S, Vtw=Vtw, alpha=alpha, ts=LL.shape,
ws=wm.shape)


def ext_pc(img, key):
    LL, _ = pywt.dwt2(img, "haar")
    if LL.shape != key["ts"]:
        LL = resize(LL, key["ts"], anti_aliasing=True, preserve_range=True)
    S1s = np.linalg.svd(dct2(LL), compute_uv=False)
    D = (key["U1"] * S1s) @ key["V1t"]
    W = ((D - np.diag(key["S"])) / key["alpha"]) @ key["Vtw"]
    return np.clip(resize(W, key["ws"], anti_aliasing=True, preserve_range=True) * 255, 0, 255)


if __name__ == "__main__":
    wm, wmB = make_watermark_logo(), make_watermark_logo(text="OTHERLOGO")
    host = np.asarray(Image.open(os.path.join(ROOT, "data", "original", "image_066.png")),
dtype=np.float64)
    other = np.asarray(Image.open(os.path.join(ROOT, "data", "original", "image_064.png")),
dtype=np.float64)
    for a in (0.1, 1.0, 5.0):
        w, k = emb_pc(host, wm, a)
        wB, kB = emb_pc(host, wmB, a)
        print(f"alpha={a:<4} PSNR={psnr(host, w):6.2f} dB | genuine NC={nc(wm, ext_pc(w, k)):.3f} | "
              f"UNwatermarked host NC={nc(wm, ext_pc(host, k)):.3f} | other image NC={nc(wm,
ext_pc(other, k)):.3f} | "
              f"logo B decoded with key A NC_A={nc(wm, ext_pc(wB, k)):.3f}")
```

Part 4: Analysis and benchmarks

Statistics, figures and comparisons built from the recorded results.

4.1 analysis.py

Tables and figures for the main results. 379 lines.

```
from __future__ import annotations

import os
os.environ.setdefault("OPENBLAS_NUM_THREADS", "1")

import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from PIL import Image
from scipy import stats

from attacks import ATTACK_FAMILY, ATTACK_NAMES, make_attack_suite
from logo import make_watermark_logo
from metrics import normalized_correlation, psnr, ssim, zero_normalized_correlation
from watermark_core import embed_watermark, extract_watermark

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
RES = os.path.join(ROOT, "results")
TAB = os.path.join(RES, "tables")
VIS = os.path.join(ROOT, "visualizations")
DATA = os.path.join(ROOT, "data")
os.makedirs(TAB, exist_ok=True)
os.makedirs(VIS, exist_ok=True)

THR = 0.75
BLUE, ORANGE, AQUA, GREY, INK = "#2a78d6", "#eb6834", "#1baf7a", "#8a8a85", "#222222"
plt.rcParams.update({
    "font.size": 10, "axes.titlesize": 11, "axes.labelsize": 10, "axes.edgecolor": "#888888",
    "axes.spines.top": False, "axes.spines.right": False, "axes.grid": True, "grid.color": "#e4e4e0",
    "grid.linewidth": 0.7, "axes.axisbelow": True, "figure.facecolor": "white", "axes.facecolor":
"white",
    "savefig.dpi": 200, "savefig.bbox": "tight", "legend.frameon": False,
})
rng = np.random.default_rng(20260713)


def wilson(k, n, z=1.96):
    if n == 0:
        return (np.nan, np.nan)
    p = k / n
    d = 1 + z * z / n
    c = (p + z * z / (2 * n)) / d
    h = z * np.sqrt(p * (1 - p) / n + z * z / (4 * n * n)) / d
    return c - h, c + h


def boot_ci(x, n=2000):
    x = np.asarray(x)
    means = rng.choice(x, size=(n, len(x)), replace=True).mean(axis=1)
    return np.percentile(means, [2.5, 97.5])


def save(fig, name):
    fig.savefig(os.path.join(VIS, name))
    plt.close(fig)


def main():
    meta = pd.read_csv(os.path.join(DATA, "metadata.csv"))
    img = pd.read_csv(os.path.join(RES, "results_images.csv"))
    att = pd.read_csv(os.path.join(RES, "results_attacks.csv"))
    sw = pd.read_csv(os.path.join(RES, "results_alpha_sweep.csv"))
    ctl = pd.read_csv(os.path.join(RES, "results_controls.csv"))
    summ = json.load(open(os.path.join(RES, "run_summary.json")))
```

```
    prim = summ["config"]["primary_protocol"]
    cat = meta.set_index("image_id")[["category", "is_synthetic", "mean_intensity", "contrast_std",
"key"]]
    out = {}

    im = img.merge(meta[["image_id", "is_synthetic", "category"]], on="image_id")
    def desc(s):
        return dict(mean=s.mean(), std=s.std(), median=s.median(), min=s.min(), max=s.max())
    out["E1"] = {
        "n_images": int(len(im)),
        "psnr_all": desc(im.psnr_db), "ssim_all": desc(im.ssim),
        "psnr_real": desc(im[im.is_synthetic == 0].psnr_db), "ssim_real": desc(im[im.is_synthetic ==
0].ssim),
        "psnr_synthetic": desc(im[im.is_synthetic == 1].psnr_db), "ssim_synthetic":
desc(im[im.is_synthetic == 1].ssim),
        "n_psnr_below_30": int((im.psnr_db < 30).sum()),
        "n_ssim_below_0.98": int((im.ssim < 0.98).sum()),
        "ssim_below_0.98_ids": list(im[im.ssim < 0.98].image_id),
        "psnr_float_mean": float(im.psnr_float_db.mean()),
        "max_abs_diff_max": float(im.max_abs_diff.max()), "mean_abs_diff_mean":
float(im.mean_abs_diff.mean()),
        "t_embed_ms_median": float(im.t_embed_ms.median()), "key_mb": float(im.key_bytes.median() /
1e6),
    }
    t1 = pd.DataFrame({"PSNR (dB)": im.psnr_db.describe(), "SSIM": im.ssim.describe()}).round(4)
    t1.to_csv(os.path.join(TAB, "imperceptibility_summary.csv"))

    a = att[att.protocol == prim].merge(cat, left_on="image_id", right_index=True)
    rows = []
    for name in ATTACK_NAMES:
        x = a[a.attack == name]
        lo, hi = boot_ci(x.nc)
        k = int((x.nc >= THR).sum())
        wl, wh = wilson(k, len(x))
        real = x[x.is_synthetic == 0]
        rows.append(dict(attack=name, family=ATTACK_FAMILY[name], n=len(x), mean_nc=x.nc.mean(),
ci_lo=lo, ci_hi=hi,
                         median_nc=x.nc.median(), std_nc=x.nc.std(), min_nc=x.nc.min(), pass_rate=k /
len(x),
                         mean_zncc=x.zncc.mean(), median_zncc=x.zncc.median(), zncc_pass_rate=(x.zncc
>= THR).mean(),
                         pass_lo=wl, pass_hi=wh, mean_ber=x.ber.mean(),
mean_nc_real_only=real.nc.mean(),
                         pass_rate_real_only=(real.nc >= THR).mean(),
t_attack_ms=x.t_attack_ms.median(),
                         t_extract_ms=x.t_extract_ms.median()))
    rob = pd.DataFrame(rows)
    rob.to_csv(os.path.join(TAB, "robustness_by_attack.csv"), index=False)
    fam = a.groupby("family").agg(mean_nc=("nc", "mean"), pass_rate=("nc", lambda s: (s >=
THR).mean()), n=("nc", "size")).reset_index()
    fam.to_csv(os.path.join(TAB, "robustness_by_family.csv"), index=False)

    other = att[att.protocol != prim].merge(cat, left_on="image_id", right_index=True)
    cmp_rows = []
    for name in ATTACK_NAMES:
        u = a[a.attack == name].set_index("image_id").nc
        f = other[other.attack == name].set_index("image_id").nc
        cmp_rows.append(dict(attack=name, mean_nc_uint8=u.mean(), mean_nc_float=f.mean(),
                             max_abs_diff_per_image=(u - f.reindex(u.index)).abs().max()))
    pd.DataFrame(cmp_rows).to_csv(os.path.join(TAB, "protocol_comparison.csv"), index=False)

    out["E2"] = {
        "protocol": prim, "n_trials": int(len(a)), "mean_nc": float(a.nc.mean()),
        "n_trials_attacks_only": int((a.attack != "No Attack").sum()),
        "n_pass_attacks_only": int(((a.attack != "No Attack") & (a.nc >= THR)).sum()),
        "pass_rate_attacks_only": float((a[a.attack != "No Attack"].nc >= THR).mean()),
        "mean_nc_attacks_only": float(a[a.attack != "No Attack"].nc.mean()),
        "pass_rate": float((a.nc >= THR).mean()), "n_pass": int((a.nc >= THR).sum()),
        "pass_rate_real_only": float((a[a.is_synthetic == 0].nc >= THR).mean()),
        "mean_nc_real_only": float(a[a.is_synthetic == 0].nc.mean()),
        "mean_nc_float_protocol": float(other.nc.mean()), "pass_rate_float_protocol": float((other.nc
>= THR).mean()),
        "mean_ber": float(a.ber.mean()), "mean_zncc": float(a.zncc.mean()), "zncc_pass_rate":
float((a.zncc >= THR).mean()),
        "zncc_clean_mean": float(a[a.attack == "No Attack"].zncc.mean()),
        "clean_nc_mean": float(a[a.attack == "No Attack"].nc.mean()), "clean_nc_min":
float(a[a.attack == "No Attack"].nc.min()),
        "attacks_with_pass_rate_ge_0.95": list(rob[rob.pass_rate >= 0.95].attack),
```

```
        "weakest": rob.sort_values("pass_rate").head(3)[["attack", "pass_rate",
"mean_nc"]].to_dict("records"),
    }

    piv = a.pivot(index="image_id", columns="attack", values="nc").join(cat)
    cor = []
    for name in ATTACK_NAMES:
        if name == "No Attack":
            continue
        r1 = stats.spearmanr(piv["contrast_std"], piv[name])
        r2 = stats.spearmanr(piv["mean_intensity"], piv[name])
        cor.append(dict(attack=name, rho_contrast=r1[0], p_contrast=r1[1], rho_mean_intensity=r2[0],
p_mean_intensity=r2[1]))
    pd.DataFrame(cor).round(4).to_csv(os.path.join(TAB, "nc_vs_image_statistics.csv"), index=False)

    ct = a.groupby(["category", "family"]).nc.mean().unstack().round(3)
    ct["n_images"] = meta.groupby("category").size()
    ct.to_csv(os.path.join(TAB, "nc_by_category_family.csv"))
    ca = a.groupby(["category", "attack"]).nc.mean().unstack().round(3)
    ca.to_csv(os.path.join(TAB, "nc_by_category_attack.csv"))

    swo = sw[sw.status == "ok"]
    s = swo.groupby(["attack", "alpha"]).agg(psnr=("psnr_db", "mean"), psnr_sd=("psnr_db", "std"),
ssim=("ssim", "mean"),
                                             nc_mean=("nc", "mean"), nc_median=("nc", "median"),
                                             nc_q25=("nc", lambda x: x.quantile(.25)), nc_q75=("nc",
lambda x: x.quantile(.75)),
                                             zncc_mean=("zncc", "mean"), ber_mean=("ber", "mean"),
                                             pass_rate=("nc", lambda x: (x >=
THR).mean())).reset_index()
    s.round(4).to_csv(os.path.join(TAB, "alpha_sweep_summary.csv"), index=False)
    out["E3"] = {"table": s.round(4).to_dict("records")}
    logo0 = make_watermark_logo(summ["config"]["logo_size"], summ["config"]["logo_text"])
    U_, S_, Vt_ = np.linalg.svd(logo0)
    rank1 = np.clip((U_[:, :1] * S_[:1]) @ Vt_[:1], 0, 255)
    out["metric_baselines"] = {
        "logo_fill_ratio": float((logo0 > 0).mean()),
        "nc_constant_white_image": normalized_correlation(logo0, np.full_like(logo0, 255.0)),
        "nc_rank1_approximation": normalized_correlation(logo0, rank1),
        "zncc_rank1_approximation": zero_normalized_correlation(logo0, rank1),
        "sigma1_energy_fraction": float(S_[0] ** 2 / (S_ ** 2).sum()),
    }

    def sel(prefix):
        return ctl[ctl.control.str.startswith(prefix)].nc
    g = sel("genuine")
    crow = []
    for lbl, pre in [("genuine: own key, clean watermarked image", "genuine"),
                     ("a: other host, UNwatermarked, key of image i", "a:"),
                     ("b: other host, watermarked, key of image i", "b:"),
                     ("c: same host, different logo embedded, decoded with key A (NC vs logo A)", "c:
"),
                     ("c-ref: same host, logo B decoded with its own key (NC vs logo B)", "c-ref"),
                     ("d: own host unwatermarked, own key (degenerate)", "d:")]:
        x = sel(pre)
        z = ctl[ctl.control.str.startswith(pre)].zncc
        crow.append(dict(control=lbl, n=len(x), mean_nc=x.mean(), median=x.median(),
p95=x.quantile(.95), max=x.max(),
                         frac_ge_0_75=(x >= THR).mean(), frac_ge_0_90=(x >= .9).mean(),
                         mean_zncc=z.mean(), p95_zncc=z.quantile(.95), max_zncc=z.max(),
frac_zncc_ge_0_75=(z >= THR).mean()))
    pd.DataFrame(crow).round(4).to_csv(os.path.join(TAB, "controls_summary.csv"), index=False)
    auc = {}
    for k, pre in [("a", "a:"), ("b", "b:"), ("c", "c: ")]:
        x = sel(pre)
        u = stats.mannwhitneyu(g, x, alternative="greater")
        auc[k] = float(u.statistic / (len(g) * len(x)))
    out["E4"] = {"summary": pd.DataFrame(crow).round(4).to_dict("records"), "auc_genuine_vs_control":
auc}

    wrong_p95 = float(pd.concat([sel("a:"), sel("b:")]).quantile(.95))
    out["E4"]["wrong_image_p95"] = wrong_p95
    out["E4"]["attacked_genuine_below_wrong_p95"] = float((a.nc < wrong_p95).mean())

    tim = a.groupby("attack").agg(t_attack_ms=("t_attack_ms", "median"),
t_extract_ms=("t_extract_ms", "median")).reset_index()
    tim.round(3).to_csv(os.path.join(TAB, "timing_by_attack.csv"), index=False)
    out["timing"] = {"embed_ms_median": float(img.t_embed_ms.median()),
```

```
                     "extract_ms_median": float(a.t_extract_ms.median()),
                     "throughput_embed_per_s_single_core": float(1000 / img.t_embed_ms.median()),
                     "peak_rss_mb": float(img.peak_rss_mb.max())}

    json.dump(out, open(os.path.join(RES, "analysis_summary.json"), "w"), indent=2, default=float)

    fig, ax = plt.subplots(1, 2, figsize=(9, 3.3))
    real = im[im.is_synthetic == 0]; syn = im[im.is_synthetic == 1]
    ax[0].hist(real.psnr_db, bins=np.arange(35.4, 44.2, 0.2), color=BLUE, label=f"real
(n={len(real)})")
    ax[0].hist(syn.psnr_db, bins=np.arange(35.4, 44.2, 0.2), color=ORANGE, label=f"synthetic
(n={len(syn)})")
    ax[0].axvline(30, color=INK, lw=1, ls="--"); ax[0].text(30.1, ax[0].get_ylim()[1] * .9, "30 dB",
fontsize=8)
    ax[0].set_xlabel("PSNR of watermarked image (dB)"); ax[0].set_ylabel("images"); ax[0].legend()
    ax[1].hist(real.ssim, bins=np.linspace(0.55, 1.0, 46), color=BLUE)
    ax[1].hist(syn.ssim, bins=np.linspace(0.55, 1.0, 46), color=ORANGE)
    ax[1].axvline(0.98, color=INK, lw=1, ls="--"); ax[1].text(0.9805, ax[1].get_ylim()[1] * .9,
"0.98", fontsize=8)
    ax[1].set_xlabel("SSIM of watermarked image"); ax[1].set_ylabel("images")
    fig.suptitle(f"Imperceptibility at alpha = {summ['config']['alpha']} across {len(im)} images",
y=1.02, fontsize=11)
    fig.tight_layout(); save(fig, "fig_imperceptibility_hist.png")

    order = rob.sort_values("median_nc")["attack"].tolist()
    fig, ax = plt.subplots(figsize=(8.6, 6.2))
    data = [a[a.attack == n].nc.values for n in order]
    bp = ax.boxplot(data, orientation="horizontal", patch_artist=True, widths=0.6, showfliers=True,
                    flierprops=dict(marker="o", markersize=2.5, markerfacecolor=GREY,
markeredgecolor="none", alpha=.6),
                    medianprops=dict(color=INK, lw=1.4), whiskerprops=dict(color=GREY),
capprops=dict(color=GREY),
                    boxprops=dict(edgecolor=GREY))
    for patch, n in zip(bp["boxes"], order):
        patch.set_facecolor(BLUE if rob.set_index("attack").loc[n, "median_nc"] >= THR else ORANGE)
        patch.set_alpha(.85)
    ax.set_yticklabels(order)
    ax.axvline(THR, color=INK, ls="--", lw=1)
    ax.text(THR - .005, len(order) + 0.55, "threshold 0.75", fontsize=8, ha="right", va="center")
    for i, n in enumerate(order, 1):
        pr = rob.set_index("attack").loc[n, "pass_rate"]
        ax.text(1.03, i, f"{pr*100:.0f}%", va="center", fontsize=8.5, color=INK)
    ax.text(1.03, len(order) + 0.55, "NC >= 0.75", fontsize=8, color=INK, va="center")
    ax.set_xlim(-0.02, 1.12); ax.set_xlabel("Normalized correlation (NC) of extracted watermark, 100
images")
    ax.set_title("Robustness per attack: NC over 100 images (box = IQR, line = median)", loc="left")
    ax.grid(axis="y", visible=False)
    fig.tight_layout(); save(fig, "fig_nc_boxplot.png")

    r = rob.sort_values("pass_rate")
    fig, ax = plt.subplots(figsize=(8, 5.4))
    ax.barh(r.attack, r.pass_rate * 100, color=[BLUE if v >= .9 else ORANGE for v in r.pass_rate],
height=.62)
    ax.errorbar(r.pass_rate * 100, np.arange(len(r)), xerr=[np.clip((r.pass_rate - r.pass_lo) * 100,
0, None), np.clip((r.pass_hi - r.pass_rate) * 100, 0, None)],
                fmt="none", ecolor=INK, elinewidth=1, capsize=2)
    for i, v in enumerate(r.pass_rate):
        ax.text(min(v * 100 + 3.5, 104), i, f"{v*100:.0f}%", va="center", fontsize=8.5)
    ax.set_xlim(0, 112); ax.set_xlabel("Images with NC >= 0.75 (%)  - error bars: 95% Wilson
interval")
    ax.set_title("Detection pass rate per attack (n = 100 images each)")
    ax.grid(axis="y", visible=False)
    fig.tight_layout(); save(fig, "fig_pass_rate.png")

    fig, ax = plt.subplots(1, 3, figsize=(11.5, 3.5))
    p0 = s[s.attack == "No Attack"]
    ax[0].plot(p0.alpha, p0.psnr, color=BLUE, marker="o", ms=4, lw=1.6)
    ax[0].fill_between(p0.alpha, p0.psnr - p0.psnr_sd, p0.psnr + p0.psnr_sd, color=BLUE, alpha=.15,
lw=0)
    ax[0].axhline(30, color=INK, lw=1, ls="--"); ax[0].set_ylabel("mean PSNR (dB)")
    ax[0].set_title("Imperceptibility (band = +-1 sd)")
    cols = {"JPEG Q=50": BLUE, "Gaussian Noise (var=0.02)": ORANGE, "Gaussian Blur (sigma=1.5)":
AQUA, "Rotation 5 deg": GREY}
    for an, c in cols.items():
        q = s[s.attack == an]
        ax[1].plot(q.alpha, q.nc_mean, color=c, marker="o", ms=3.5, lw=1.5, label=an)
        ax[2].plot(q.alpha, q.pass_rate * 100, color=c, marker="o", ms=3.5, lw=1.5, label=an)
```

```
    ax[1].axhline(THR, color=INK, lw=1, ls="--"); ax[1].set_ylim(0.4, 1.02); ax[1].set_ylabel("mean
NC (100 images)")
    ax[1].set_title("Robustness: mean NC after attack"); ax[1].legend(fontsize=7.5, loc="lower
right")
    ax[2].set_ylim(0, 102); ax[2].set_ylabel("images with NC >= 0.75 (%)");
ax[2].set_title("Robustness: detection pass rate")
    for x_ in ax:
        x_.set_xlabel("embedding strength alpha")
    fig.tight_layout(); save(fig, "fig_alpha_tradeoff.png")

    fig, ax = plt.subplots(figsize=(8, 5.2))
    rr = rob.sort_values("mean_nc")
    yy = np.arange(len(rr))
    ax.barh(yy + .18, rr.mean_nc, height=.34, color=BLUE, label="NC (original metric)")
    ax.barh(yy - .18, rr.mean_zncc, height=.34, color=ORANGE, label="ZNCC (mean-centred)")
    ax.set_yticks(yy); ax.set_yticklabels(rr.attack); ax.axvline(THR, color=INK, ls="--", lw=1)
    ax.set_xlabel("mean over 100 images"); ax.legend(loc="lower right"); ax.grid(axis="y",
visible=False)
    ax.set_title("Plain NC is optimistic: ZNCC removes the brightness offset")
    fig.tight_layout(); save(fig, "fig_nc_vs_zncc.png")

    fig, ax = plt.subplots(figsize=(8, 4))
    bins = np.linspace(0, 1, 41)
    ax.hist(sel("a:"), bins=bins, alpha=.85, color=ORANGE, label="a: other host, unwatermarked",
density=True)
    ax.hist(sel("b:"), bins=bins, alpha=.55, color=AQUA, label="b: other host, watermarked (wrong
key)", density=True)
    ax.hist(g, bins=bins, alpha=.75, color=BLUE, label="genuine: own key", density=True)
    ax.axvline(THR, color=INK, ls="--", lw=1)
    ax.set_xlabel("NC against the owner's logo"); ax.set_ylabel("density")
    ax.set_title("Control experiments: wrong-image/wrong-key NC overlaps the 0.75 threshold")
    ax.legend(loc="upper center", fontsize=8)
    fig.tight_layout(); save(fig, "fig_controls_hist.png")

    cc = sel("c: "); cr = sel("c-ref")
    fig, ax = plt.subplots(figsize=(6.4, 3.4))
    vals = [g.mean(), cc.mean(), cr.mean(), sel("a:").mean(), sel("b:").mean()]
    labs = ["genuine\n(logo A, key A)", "logo B embedded,\ndecoded with key A", "logo B embedded,\
ndecoded with key B", "other host,\nunwatermarked", "other host,\nwrong key"]
    ax.bar(labs, vals, color=[BLUE, ORANGE, BLUE, GREY, GREY], width=.6)
    for i, v in enumerate(vals):
        ax.text(i, v + .015, f"{v:.3f}", ha="center", fontsize=9)
    ax.set_ylim(0, 1.12); ax.set_ylabel("mean NC vs the logo tested")
    ax.set_title("False-positive problem: the wrong logo is reported as the owner's logo")
    ax.tick_params(axis="x", labelsize=8); ax.grid(axis="x", visible=False)
    fig.tight_layout(); save(fig, "fig_false_positive.png")

    hm = ct.drop(columns="n_images")
    hm = hm.loc[hm.index != "synthetic_control"]
    fig, ax = plt.subplots(figsize=(8.4, 6.2))
    im_ = ax.imshow(hm.values, cmap="Blues", vmin=0, vmax=1, aspect="auto")
    ax.set_xticks(range(hm.shape[1])); ax.set_xticklabels(hm.columns, rotation=30, ha="right")
    ax.set_yticks(range(hm.shape[0])); ax.set_yticklabels([f"{c} ({int(ct.loc[c,'n_images'])})" for c
in hm.index])
    for i in range(hm.shape[0]):
        for j in range(hm.shape[1]):
            v = hm.values[i, j]
            ax.text(j, i, f"{v:.2f}", ha="center", va="center", fontsize=8, color="white" if v > .62
else INK)
    ax.grid(False); fig.colorbar(im_, ax=ax, fraction=.04, label="mean NC")
    ax.set_title("Mean NC by image category and attack family (real images)")
    fig.tight_layout(); save(fig, "fig_category_heatmap.png")

    fig, ax = plt.subplots(1, 3, figsize=(10.5, 3.3), sharey=True)
    for k, name in enumerate(["Gaussian Noise (var=0.02)", "Rotation 15 deg", "Histogram
Equalization"]):
        d = piv
        ax[k].scatter(d["contrast_std"], d[name], s=14, c=np.where(d["is_synthetic"] == 1, ORANGE,
BLUE), alpha=.8, lw=0)
        rho = stats.spearmanr(d["contrast_std"], d[name])[0]
        ax[k].set_title(f"{name}\nSpearman rho = {rho:.2f}", fontsize=9.5)
        ax[k].set_xlabel("image contrast (std of grey levels)"); ax[k].axhline(THR, color=INK,
ls="--", lw=1)
    ax[0].set_ylabel("NC after attack")
    fig.tight_layout(); save(fig, "fig_nc_vs_contrast.png")

    fig, ax = plt.subplots(figsize=(6.5, 3.2))
```

```
    ax.hist(img.t_embed_ms, bins=20, color=BLUE, label=f"embed (median {img.t_embed_ms.median():.0f}
ms)")
    ax.hist(a.t_extract_ms, bins=40, color=ORANGE, alpha=.8, label=f"extract (median
{a.t_extract_ms.median():.0f} ms)")
    ax.set_xlabel("time per 512x512 image (ms, single thread)"); ax.set_ylabel("operations");
ax.legend()
    fig.tight_layout(); save(fig, "fig_timing.png")

    cfg = summ["config"]; alpha = cfg["alpha"]
    logo = make_watermark_logo(cfg["logo_size"], cfg["logo_text"])
    Image.fromarray(logo.astype(np.uint8)).save(os.path.join(VIS, "watermark_logo.png"))
    reps = ["image_066", "image_064", "image_001", "image_051", "image_055", "image_100"]
    fig, axes = plt.subplots(len(reps), 3, figsize=(8.4, 2.8 * len(reps)))
    for r_i, iid in enumerate(reps):
        host = np.asarray(Image.open(os.path.join(DATA, "original", iid + ".png")), dtype=np.float64)
        w = np.asarray(Image.open(os.path.join(ROOT, "outputs", "watermarked", iid + "_wm.png")),
dtype=np.float64)
        d = np.abs(host - w)
        row = img.set_index("image_id").loc[iid]
        key = meta.set_index("image_id").loc[iid, "key"]
        axes[r_i, 0].imshow(host, cmap="gray", vmin=0, vmax=255); axes[r_i, 0].set_title(f"{iid}
({key}) original", fontsize=8.5)
        axes[r_i, 1].imshow(w, cmap="gray", vmin=0, vmax=255); axes[r_i, 1].set_title(f"watermarked
PSNR {row.psnr_db:.1f} dB, SSIM {row.ssim:.3f}", fontsize=8.5)
        im2 = axes[r_i, 2].imshow(np.clip(d * 10, 0, 255), cmap="magma", vmin=0, vmax=255); axes[r_i,
2].set_title(f"|difference| x10 (max {d.max():.0f} grey levels)", fontsize=8.5)
        for x in axes[r_i]:
            x.axis("off")
    fig.tight_layout(); save(fig, "fig_representative_comparison.png")

    iid = "image_066"
    host = np.asarray(Image.open(os.path.join(DATA, "original", iid + ".png")), dtype=np.float64)
    w, key = embed_watermark(host, logo, alpha=alpha)
    wq = np.clip(np.rint(w), 0, 255)
    suite = make_attack_suite(cfg["base_seed"] + 100 * (int(iid[-3:]) - 1))
    fig, axes = plt.subplots(3, 6, figsize=(12, 6.6))
    axes[0, 0].imshow(logo, cmap="gray"); axes[0, 0].set_title("original logo", fontsize=8.5)
    for ax_, name in zip(axes.flat[1:], ATTACK_NAMES):
        ext = extract_watermark(suite[name](wq), key)
        ax_.imshow(ext, cmap="gray", vmin=0, vmax=255)
        ax_.set_title(f"{name}\nNC={normalized_correlation(logo, ext):.3f}", fontsize=7.8)
    for ax_ in axes.flat[16:]:
        ax_.axis("off")
    for ax_ in axes.flat:
        ax_.set_xticks([]); ax_.set_yticks([]); ax_.grid(False)
    fig.suptitle("Extracted watermark under each attack (image_066, cameraman)", y=1.0, fontsize=11)
    fig.tight_layout(); save(fig, "fig_extraction_gallery.png")

    T = 128
    sheet = Image.new("L", (10 * T, 10 * T), 255)
    for i, iid in enumerate(meta.image_id):
        sheet.paste(Image.open(os.path.join(ROOT, "outputs", "watermarked", iid +
"_wm.png")).resize((T, T)), ((i % 10) * T, (i // 10) * T))
    sheet.save(os.path.join(VIS, "contact_sheet_watermarked.png"))
    print(json.dumps({k: out[k] for k in ("E1", "E2")}, indent=1, default=float)[:3000])


if __name__ == "__main__":
    main()
```

4.2 analysis_tech.py

Figures for the technology comparison. 71 lines.

```
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
RES, VIS = os.path.join(ROOT, "results"), os.path.join(ROOT, "visualizations")
BLUE, ORANGE, AQUA, GREY, INK = "#2a78d6", "#eb6834", "#1baf7a", "#8a8a85", "#222222"
plt.rcParams.update({"font.size": 10, "axes.spines.top": False, "axes.spines.right": False,
"axes.grid": True,
```

```
                    "grid.color": "#e4e4e0", "axes.axisbelow": True, "savefig.dpi": 200,
"savefig.bbox": "tight",
                    "legend.frameon": False})

A = pd.read_csv(os.path.join(RES, "tech_benchmark_components.csv"))
fig, ax = plt.subplots(figsize=(9, 8))
labels, vals, cols, ypos = [], [], [], []
y = 0
for op, g in A.groupby("operation", sort=False):
    ax.text(0.0008, y - 0.1, op, fontsize=9, fontweight="bold", va="bottom",
transform=ax.get_yaxis_transform() if False else ax.transData)
    y += 0.9
    for _, r in g.iterrows():
        labels.append(r.implementation); vals.append(r.median_ms)
        cols.append(BLUE if "(project)" in r.implementation else (ORANGE if r.includes_process_and_io
else GREY))
        ypos.append(y); y += 1
    y += 0.5
ax.barh(ypos, vals, color=cols, height=.7)
ax.set_yticks(ypos); ax.set_yticklabels(labels, fontsize=8); ax.invert_yaxis(); ax.set_xscale("log")
for yy, v in zip(ypos, vals):
    ax.text(v * 1.08, yy, f"{v:.2f} ms", va="center", fontsize=7.5)
ax.set_xlim(0.03, 400); ax.set_xlabel("median time per call, ms (log scale; single thread, in-
memory)"); ax.grid(axis="y", visible=False)
ax.set_title("Same operation, different libraries (blue = implementation used by the project;\norange
= includes process start-up and file I/O)", fontsize=10)
fig.tight_layout(); fig.savefig(os.path.join(VIS, "fig_tech_component_speed.png")); plt.close(fig)

S = pd.read_csv(os.path.join(RES, "tech_benchmark_schemes.csv"))
atts = ["No Attack", "JPEG Q=50", "JPEG Q=20", "Gaussian Noise (var=0.02)", "Median Filter 3x3",
"Gaussian Blur (sigma=1.5)",
        "Rotation 5 deg", "Cropping 25%", "Scaling 0.5x"]
order = ["Project: global DWT-DCT-SVD", "blind-watermark 0.4.4 (block DWT-DCT-SVD)", "Baseline: 8x8
block DCT", "Baseline: spatial LSB"]
cols = {order[0]: BLUE, order[1]: ORANGE, order[2]: AQUA, order[3]: GREY}
fig, ax = plt.subplots(figsize=(10, 4.2))
w = 0.2
for i, s in enumerate(order):
    d = S[S.scheme == s].set_index("attack").reindex(atts)
    ax.bar(np.arange(len(atts)) + (i - 1.5) * w, d.mean_ber, width=w * .92, color=cols[s],
           label=f"{s}  (PSNR {d.psnr_db.mean():.1f} dB)")
ax.axhline(0.5, color=INK, lw=.8, ls=":"); ax.text(len(atts) - .5, 0.505, "0.5 = random guess",
ha="right", fontsize=8)
ax.set_xticks(range(len(atts))); ax.set_xticklabels([a.replace(" (", "\n(") for a in atts],
fontsize=8)
ax.set_ylabel("mean bit-error rate of recovered logo (30 images)"); ax.set_ylim(0, .6)
ax.legend(fontsize=8, loc="upper left", bbox_to_anchor=(0, 1.32), ncol=2)
ax.set_title("Same 4095-bit payload, same attacks", y=1.0, loc="right", fontsize=9)
fig.tight_layout(); fig.savefig(os.path.join(VIS, "fig_tech_scheme_ber.png")); plt.close(fig)
print("ok")

best = (A[~A.includes_process_and_io].sort_values("speed_vs_project",
ascending=False).groupby("operation", sort=False).head(1)
        .sort_values("speed_vs_project"))
fig, ax = plt.subplots(figsize=(8.2, 4.6))
y = np.arange(len(best))
cols_ = [BLUE if v > 1.5 else GREY for v in best.speed_vs_project]
ax.barh(y, best.speed_vs_project, color=cols_, height=.62)
ax.set_xscale("log"); ax.set_xlim(0.7, 1500)
ax.axvline(1, color=INK, lw=1)
ax.set_yticks(y); ax.set_yticklabels([o.replace(", 512x512", "").replace(" 512x512", "").replace("
256x256", "") for o in best.operation], fontsize=9.5)
for yy, (_, r) in zip(y, best.iterrows()):
    lab = "project's own library is fastest" if "(project)" in r.implementation else
r.implementation.replace(" (project)", "")
    ax.text(r.speed_vs_project * 1.12, yy, f"{r.speed_vs_project:.1f}×  {lab}", va="center",
fontsize=8.8)
ax.set_xlabel("speed-up of the fastest alternative vs the project's implementation (× , log scale)")
ax.set_title("Best alternative per operation (single thread, in-memory; outputs identical or ≥ 48 dB
PSNR apart)", fontsize=10.5)
ax.grid(axis="y", visible=False)
fig.tight_layout(); fig.savefig(os.path.join(VIS, "fig_tech_speedup.png")); plt.close(fig)
print("speedup ok")
```

4.3 benchmark_tech.py

Library and scheme benchmarks. 433 lines.

```
from __future__ import annotations

import os
for _v in ("OPENBLAS_NUM_THREADS", "OMP_NUM_THREADS", "MKL_NUM_THREADS"):
    os.environ.setdefault(_v, "1")

import io
import json
import shutil
import subprocess
import sys
import tempfile
import time
import warnings

import cv2
import numpy as np
import pandas as pd
import pywt
import scipy
import scipy.fft
import scipy.linalg
import skimage
from PIL import Image, ImageFilter
from scipy import ndimage
from scipy.fftpack import dct as _dct
from skimage import exposure, filters as skfilters, transform as sktransform
from skimage.metrics import peak_signal_noise_ratio as sk_psnr

from attacks import make_attack_suite
from logo import make_watermark_logo
from metrics import psnr
from watermark_core import dct2, embed_watermark, extract_watermark

warnings.filterwarnings("ignore")
cv2.setNumThreads(1)
HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.abspath(os.path.join(HERE, "..", ".."))
RES = os.path.join(ROOT, "results")
VIS = os.path.join(ROOT, "visualizations")
DATA = os.path.join(ROOT, "data")


def load(iid):
    return np.asarray(Image.open(os.path.join(DATA, "original", iid + ".png")), dtype=np.float64)


def tmed(fn, repeats=9, warm=2):
    for _ in range(warm):
        fn()
    ts = []
    for _ in range(repeats):
        t = time.perf_counter()
        fn()
        ts.append((time.perf_counter() - t) * 1e3)
    return float(np.median(ts)), float(np.percentile(ts, 25)), float(np.percentile(ts, 75))


def have(cmd):
    return shutil.which(cmd) is not None


def dct_matrix(n):
    k = np.arange(n)[:, None]; i = np.arange(n)[None, :]
    c = np.sqrt(2.0 / n) * np.cos(np.pi * (2 * i + 1) * k / (2 * n))
    c[0] /= np.sqrt(2)
    return c


def haar_numpy(a):
    a00, a01, a10, a11 = a[0::2, 0::2], a[0::2, 1::2], a[1::2, 0::2], a[1::2, 1::2]
    return (a00 + a01 + a10 + a11) / 2.0


def jpeg_pillow(a, q=50):
    b = io.BytesIO(); Image.fromarray(a.astype(np.uint8)).save(b, "JPEG", quality=q); n = b.tell();
b.seek(0)
    return np.array(Image.open(b)).astype(np.float64), n
```

```
def jpeg_cv2(a, q=50):
    ok, buf = cv2.imencode(".jpg", a.astype(np.uint8), [cv2.IMWRITE_JPEG_QUALITY, q])
    return cv2.imdecode(buf, cv2.IMREAD_GRAYSCALE).astype(np.float64), len(buf)


def jpeg_magick(a, q=50, tmp=None):
    src, dst = os.path.join(tmp, "in.png"), os.path.join(tmp, "out.jpg")
    Image.fromarray(a.astype(np.uint8)).save(src)
    subprocess.run(["convert", src, "-quality", str(q), dst], check=True)
    return np.array(Image.open(dst).convert("L")).astype(np.float64), os.path.getsize(dst)


def rot_skimage(a):
    return sktransform.rotate(a / 255.0, 15, resize=False, mode="edge") * 255.0


def rot_cv2(a):
    h, w = a.shape
    M = cv2.getRotationMatrix2D(((w - 1) / 2, (h - 1) / 2), 15, 1.0)
    return cv2.warpAffine(a, M, (w, h), flags=cv2.INTER_LINEAR, borderMode=cv2.BORDER_REPLICATE)


def rot_pillow(a):
    return np.array(Image.fromarray(a.astype(np.uint8)).rotate(15,
resample=Image.BILINEAR)).astype(np.float64)


def scale_skimage(a):
    s = sktransform.resize(a, (a.shape[0] // 2, a.shape[1] // 2), anti_aliasing=True,
preserve_range=True)
    return sktransform.resize(s, a.shape, anti_aliasing=True, preserve_range=True)


def scale_cv2(a):
    s = cv2.resize(a, (a.shape[1] // 2, a.shape[0] // 2), interpolation=cv2.INTER_AREA)
    return cv2.resize(s, (a.shape[1], a.shape[0]), interpolation=cv2.INTER_LINEAR)


def scale_pillow(a):
    im = Image.fromarray(a.astype(np.uint8))
    s = im.resize((a.shape[1] // 2, a.shape[0] // 2), Image.BILINEAR, reducing_gap=None)
    return np.array(s.resize(im.size, Image.BILINEAR)).astype(np.float64)


def part_a(images):
    tmp = tempfile.mkdtemp()
    rows = []
    h512 = images[0]
    ll = h512[:256, :256].copy()
    u8 = h512.astype(np.uint8)
    C = dct_matrix(256)

    ops = {
        "2-D DCT-II 256x256": [
            ("scipy.fftpack.dct (project)", lambda: dct2(ll), "ref"),
            ("scipy.fft.dctn", lambda: scipy.fft.dctn(ll, norm="ortho"), ""),
            ("cv2.dct", lambda: cv2.dct(ll), ""),
            ("NumPy matrix C.A.C^T", lambda: C @ ll @ C.T, ""),
        ],
        "SVD 256x256": [
            ("numpy.linalg.svd (project)", lambda: np.linalg.svd(ll, full_matrices=False)[1], "ref"),
            ("scipy.linalg.svd (gesdd)", lambda: scipy.linalg.svd(ll, full_matrices=False,
check_finite=False)[1], ""),
            ("scipy.linalg.svd (gesvd)", lambda: scipy.linalg.svd(ll, full_matrices=False,
lapack_driver="gesvd", check_finite=False)[1], ""),
            ("cv2.SVDecomp", lambda: cv2.SVDecomp(ll)[0].ravel(), ""),
        ],
        "Haar DWT level-1 (LL) 512x512": [
            ("PyWavelets dwt2 (project)", lambda: pywt.dwt2(h512, "haar")[0], "ref"),
            ("NumPy slicing", lambda: haar_numpy(h512), ""),
        ],
        "JPEG round-trip Q=50, 512x512": [
            ("Pillow (project)", lambda: jpeg_pillow(h512)[0], "ref"),
            ("OpenCV imencode/imdecode", lambda: jpeg_cv2(h512)[0], ""),
        ] + ([("ImageMagick convert (process + file I/O)", lambda: jpeg_magick(h512, tmp=tmp)[0],
"io")] if have("convert") else []),
```

```
        "Median filter 3x3": [
            ("scipy.ndimage (project)", lambda: ndimage.median_filter(h512, size=3), "ref"),
            ("cv2.medianBlur (uint8)", lambda: cv2.medianBlur(u8, 3), ""),
            ("skimage.filters.median (uint8)", lambda: skfilters.median(u8, np.ones((3, 3), bool)),
""),
        ],
        "Gaussian blur sigma=1.5": [
            ("scipy.ndimage (project)", lambda: ndimage.gaussian_filter(h512, 1.5), "ref"),
            ("cv2.GaussianBlur", lambda: cv2.GaussianBlur(h512, (0, 0), 1.5,
borderType=cv2.BORDER_REFLECT), ""),
            ("skimage.filters.gaussian", lambda: skfilters.gaussian(h512, sigma=1.5,
preserve_range=True), ""),
        ],
        "Rotation 15 deg (bilinear)": [
            ("skimage.transform.rotate (project)", lambda: rot_skimage(h512), "ref"),
            ("cv2.warpAffine", lambda: rot_cv2(h512), ""),
            ("Pillow Image.rotate", lambda: rot_pillow(h512), ""),
        ],
        "Down/up-scale 0.5x": [
            ("skimage.transform.resize (project)", lambda: scale_skimage(h512), "ref"),
            ("cv2.resize (AREA down, LINEAR up)", lambda: scale_cv2(h512), ""),
            ("Pillow resize (BILINEAR)", lambda: scale_pillow(h512), ""),
        ],
        "Histogram equalisation": [
            ("skimage.exposure.equalize_hist (project)", lambda: exposure.equalize_hist(h512 / 255.0)
* 255.0, "ref"),
            ("cv2.equalizeHist (uint8)", lambda: cv2.equalizeHist(u8).astype(np.float64), ""),
        ],
        "PSNR 512x512": [
            ("skimage.metrics (project)", lambda: sk_psnr(h512, h512 + 1.0, data_range=255), "ref"),
            ("cv2.PSNR", lambda: cv2.PSNR(h512, h512 + 1.0), ""),
            ("NumPy formula", lambda: 10 * np.log10(255 ** 2 / np.mean((h512 - (h512 + 1.0)) ** 2)),
""),
        ],
    }
    for op, variants in ops.items():
        ref = np.asarray(variants[0][1]())
        for label, fn, tag in variants:
            out = np.asarray(fn())
            med, q1, q3 = tmed(fn)
            row = dict(operation=op, implementation=label, median_ms=med, q25_ms=q1, q75_ms=q3,
                       includes_process_and_io=(tag == "io"))
            if out.shape == ref.shape and out.size > 1:
                if op.startswith("Rotation"):
                    sl = (slice(128, 384), slice(128, 384))
                    d = out[sl] - ref[sl]
                    row["agreement"] = "PSNR %.1f dB (interior)" % (10 * np.log10(255 ** 2 /
max(np.mean(d ** 2), 1e-12)))
                elif op.startswith("SVD") or op.startswith("2-D DCT"):
                    scale = max(np.abs(ref).max(), 1e-12)
                    row["agreement"] = "max rel. diff %.1e" % (np.abs(out - ref).max() / scale)
                elif op.startswith("Haar"):
                    row["agreement"] = "max abs diff %.1e" % np.abs(out - ref).max()
                else:
                    d = out.astype(np.float64) - ref.astype(np.float64)
                    mse = np.mean(d ** 2)
                    row["agreement"] = "identical" if mse == 0 else "PSNR %.1f dB" % (10 *
np.log10(255 ** 2 / mse))
            else:
                row["agreement"] = "%.4f vs %.4f" % (float(np.ravel(out)[0]), float(np.ravel(ref)
[0]))
            rows.append(row)
    j = {"Pillow": jpeg_pillow(h512)[1], "OpenCV": jpeg_cv2(h512)[1]}
    if have("convert"):
        j["ImageMagick"] = jpeg_magick(h512, tmp=tmp)[1]
    shutil.rmtree(tmp, ignore_errors=True)
    return pd.DataFrame(rows), j


NB = 4095


def bits_to_logo(bits):
    b = np.zeros(4096); b[:NB] = np.asarray(bits[:NB], dtype=float)
    return b.reshape(64, 64) * 255.0


class Scheme:
```

```
    name = ""
    blind = None
    needs = ""


class OursScheme(Scheme):
    name = "Project: global DWT-DCT-SVD"
    blind = False
    needs = "owner's key (S_orig, Uw, Vw), ~2.1 MB per image"

    def __init__(self, alpha=0.10):
        self.alpha = alpha
        self.logo = make_watermark_logo()

    def embed(self, host):
        w, self.key = embed_watermark(host, self.logo, alpha=self.alpha)
        return w

    def extract(self, img):
        return extract_watermark(img, self.key) > 127.5


class LSBScheme(Scheme):
    name = "Baseline: spatial LSB"
    blind = True
    needs = "payload length only"

    def __init__(self, bits):
        self.bits = np.asarray(bits[:NB], dtype=np.uint8)

    def embed(self, host):
        flat = np.rint(host).astype(np.uint8).flatten()
        flat[:NB] = (flat[:NB] & 0xFE) | self.bits
        return flat.reshape(host.shape).astype(np.float64)

    def extract(self, img):
        flat = np.clip(np.rint(img), 0, 255).astype(np.uint8).flatten()
        return bits_to_logo(flat[:NB] & 1) > 127.5


class BlockDCTScheme(Scheme):
    name = "Baseline: 8x8 block DCT"
    blind = True
    needs = "payload length only"

    def __init__(self, bits, delta=24.0):
        self.bits = np.asarray(bits[:NB], dtype=np.uint8)
        self.delta = delta
        n = 8
        k = np.arange(n)[:, None]; i = np.arange(n)[None, :]
        c = np.sqrt(2.0 / n) * np.cos(np.pi * (2 * i + 1) * k / (2 * n)); c[0] /= np.sqrt(2)
        self.C = c

    def embed(self, host):
        out = host.astype(np.float64).copy()
        H, W = host.shape
        idx = 0
        for by in range(0, H, 8):
            for bx in range(0, W, 8):
                if idx >= NB:
                    break
                blk = out[by:by + 8, bx:bx + 8]
                d = self.C @ blk @ self.C.T
                a, b = d[4, 3], d[3, 4]
                m = (a + b) / 2
                if self.bits[idx]:
                    d[4, 3], d[3, 4] = m + self.delta / 2, m - self.delta / 2
                else:
                    d[4, 3], d[3, 4] = m - self.delta / 2, m + self.delta / 2
                out[by:by + 8, bx:bx + 8] = self.C.T @ d @ self.C
                idx += 1
        return np.clip(out, 0, 255)

    def extract(self, img):
        H, W = img.shape
        bits = []
        for by in range(0, H, 8):
            for bx in range(0, W, 8):
```

```
                if len(bits) >= NB:
                    break
                d = self.C @ img[by:by + 8, bx:bx + 8] @ self.C.T
                bits.append(1 if d[4, 3] > d[3, 4] else 0)
        return bits_to_logo(bits) > 127.5


class BlindWMScheme(Scheme):
    name = "blind-watermark 0.4.4 (block DWT-DCT-SVD)"
    blind = True
    needs = "two integer passwords + payload length"

    def __init__(self, bits):
        from blind_watermark import WaterMark
        self.WaterMark = WaterMark
        self.bits = np.asarray(bits[:NB], dtype=bool)

    def embed(self, host):
        bwm = self.WaterMark(password_img=1, password_wm=1, processes=1)
        bwm.read_img(img=np.dstack([host.astype(np.uint8)] * 3))
        bwm.read_wm(self.bits, mode="bit")
        e = bwm.embed()
        return np.clip(np.rint(e), 0, 255)

    def extract(self, img3):
        bwm = self.WaterMark(password_img=1, password_wm=1, processes=1)
        ex = bwm.extract(embed_img=np.clip(np.rint(img3), 0, 255).astype(np.uint8), wm_shape=NB,
mode="bit")
        return bits_to_logo(np.asarray(ex) > 0.5) > 127.5


def channelwise(fn, img):
    if img.ndim == 2:
        return fn(img)
    return np.stack([fn(img[..., c]) for c in range(img.shape[2])], axis=-1)


def luma(img):
    return img if img.ndim == 2 else cv2.cvtColor(img.astype(np.float32),
cv2.COLOR_BGR2GRAY).astype(np.float64)


def part_b(ids, attacks):
    logo = make_watermark_logo()
    bits = (logo.flatten() > 127)[:NB]
    ref_logo = bits_to_logo(bits)
    rows = []
    for k, iid in enumerate(ids):
        host = load(iid)
        suite = make_attack_suite(777 + k)
        schemes = [OursScheme(), BlindWMScheme(bits), LSBScheme(bits), BlockDCTScheme(bits)]
        for sch in schemes:
            try:
                t0 = time.perf_counter(); w = sch.embed(host); t_emb = (time.perf_counter() - t0) *
1e3
                wq = np.clip(np.rint(w), 0, 255)
                p = psnr(host, luma(wq))
            except Exception as exc:
                rows.append(dict(image_id=iid, scheme=sch.name, attack="(embed)", status=f"error:
{exc}"))
                continue
            for an in attacks:
                row = dict(image_id=iid, scheme=sch.name, attack=an, psnr_db=p, t_embed_ms=t_emb,
status="ok")
                try:
                    att = channelwise(suite[an], wq)
                    t0 = time.perf_counter(); ex = sch.extract(att); row["t_extract_ms"] =
(time.perf_counter() - t0) * 1e3
                    ex_img = ex.astype(np.float64) * 255.0
                    row["ber"] = float(np.mean((ex_img > 127.5) != (ref_logo > 127.5)))
                    a, b = ref_logo.flatten(), ex_img.flatten()
                    den = np.sqrt((a ** 2).sum()) * np.sqrt((b ** 2).sum())
                    row["nc"] = float((a * b).sum() / den) if den else 0.0
                except Exception as exc:
                    row.update(status=f"error: {type(exc).__name__}: {exc}")
                rows.append(row)
        print("scheme comparison: image", k + 1, "/", len(ids), flush=True)
    return pd.DataFrame(rows)
```

```
def tune_delta(ids):
    logo = make_watermark_logo(); bits = (logo.flatten() > 127)[:NB]
    target = np.mean([psnr(load(i), OursScheme().embed(load(i))) for i in ids])
    best = None
    for d in (8, 12, 16, 20, 24, 28, 32, 40, 48):
        p = np.mean([psnr(load(i), np.rint(BlockDCTScheme(bits, d).embed(load(i)))) for i in ids])
        if best is None or abs(p - target) < abs(best[1] - target):
            best = (d, p)
    return best, target


def main():
    os.makedirs(RES, exist_ok=True)
    meta = pd.read_csv(os.path.join(DATA, "metadata.csv"))
    real = meta[meta.is_synthetic == 0].image_id.tolist()
    rng = np.random.default_rng(20260713)

    comp_ids = list(rng.choice(real, 3, replace=False))
    parts = []
    for iid in comp_ids:
        df, jsz = part_a([load(iid)])
        df["image_id"] = iid
        parts.append(df)
    A = pd.concat(parts)
    Agg = A.groupby(["operation", "implementation"], sort=False).agg(
        median_ms=("median_ms", "median"), q25_ms=("q25_ms", "median"), q75_ms=("q75_ms", "median"),
        includes_process_and_io=("includes_process_and_io", "first"), agreement=("agreement",
"first")).reset_index()
    ref = Agg.groupby("operation", sort=False).median_ms.transform("first")
    Agg["speed_vs_project"] = ref / Agg.median_ms
    Agg.round(4).to_csv(os.path.join(RES, "tech_benchmark_components.csv"), index=False)
    print(Agg.round(3).to_string())

    sample = list(rng.choice(real, 30, replace=False))
    tune_ids = sample[:5]
    (delta, p_d), p_target = tune_delta(tune_ids)
    print("block-DCT delta tuned to", delta, "PSNR", round(p_d, 2), "target", round(p_target, 2))
    BlockDCTScheme.__init__.__defaults__ = (delta,)
    attacks = ["No Attack", "JPEG Q=50", "JPEG Q=20", "Gaussian Noise (var=0.02)", "Median Filter
3x3",
               "Gaussian Blur (sigma=1.5)", "Rotation 5 deg", "Cropping 25%", "Scaling 0.5x"]
    B = part_b(sample, attacks)
    B.to_csv(os.path.join(RES, "tech_benchmark_schemes_raw.csv"), index=False)
    ok = B[B.status == "ok"]
    S = ok.groupby(["scheme", "attack"], sort=False).agg(mean_ber=("ber", "mean"), median_ber=("ber",
"median"),
                                                         mean_nc=("nc", "mean"), psnr_db=("psnr_db",
"mean"),
                                                         t_embed_ms=("t_embed_ms", "median"),
t_extract_ms=("t_extract_ms", "median"),
                                                         ber_le_0_10=("ber", lambda x: (x <=
0.10).mean()), n=("ber", "size")).reset_index()
    S.round(4).to_csv(os.path.join(RES, "tech_benchmark_schemes.csv"), index=False)
    print(S.round(3).to_string())

    env = {"python": sys.version.split()[0], "numpy": np.__version__, "scipy": scipy.__version__,
"scikit-image": skimage.__version__,
           "opencv": cv2.__version__, "Pillow": __import__("PIL").__version__, "PyWavelets":
pywt.__version__,
           "blind-watermark": __import__("importlib.metadata", fromlist=["version"]).version("blind-
watermark"),
           "ImageMagick": (subprocess.run(["convert", "-version"], capture_output=True,
text=True).stdout.splitlines() or [""])[0] if have("convert") else None,
           "ffmpeg": (subprocess.run(["ffmpeg", "-version"], capture_output=True,
text=True).stdout.splitlines() or [""])[0] if have("ffmpeg") else None,
           "cpu_threads_used": 1, "repeats": 9, "warmup": 2, "component_images": comp_ids,
"scheme_images": sample,
           "jpeg_bytes_example": jsz, "blockdct_delta": delta, "blockdct_psnr": p_d,
"project_psnr_tune": p_target}
    json.dump(env, open(os.path.join(RES, "tech_summary.json"), "w"), indent=2, default=str)


if __name__ == "__main__":
    main()
```

4.4 make_diagram.py

Pipeline diagrams. 98 lines.

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch
import os

RESULTS_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "..", "..", "visualizations")


def box(ax, xy, w, h, text, fc="#eaf2f8", ec="#2874a6", fontsize=9.5):
    x, y = xy
    p = FancyBboxPatch((x, y), w, h, boxstyle="round,pad=0.02,rounding_size=0.02",
                       fc=fc, ec=ec, linewidth=1.6)
    ax.add_patch(p)
    ax.text(x + w / 2, y + h / 2, text, ha="center", va="center",
            fontsize=fontsize, wrap=True)


def arrow(ax, p1, p2, color="#333333"):
    a = FancyArrowPatch(p1, p2, arrowstyle="-|>", mutation_scale=14,
                        color=color, linewidth=1.5)
    ax.add_patch(a)


def embedding_diagram():
    fig, ax = plt.subplots(figsize=(11, 4.2))
    ax.set_xlim(0, 11); ax.set_ylim(0, 4.2); ax.axis("off")
    ax.set_title("Watermark Embedding Pipeline", fontsize=13, fontweight="bold", pad=16)

    steps = [
        "Host Image\n(512x512, grayscale)",
        "2D DWT\n(Haar, 1-level)\n-> LL sub-band",
        "2-D DCT\nof LL sub-band\n(global)",
        "SVD\nA = U.S.V^T",
        "S' = S + alpha*Sw\n(Sw: singular\nvalues of logo)",
        "Inverse DCT\n+ Inverse DWT",
        "Watermarked\nImage",
    ]
    n = len(steps)
    w, h = 1.35, 1.5
    gap = (11 - n * w) / (n + 1)
    x = gap
    y = 1.1
    centers = []
    for s in steps:
        box(ax, (x, y), w, h, s, fontsize=9.2)
        centers.append((x, x + w))
        x += w + gap
    for i in range(n - 1):
        arrow(ax, (centers[i][1], y + h / 2), (centers[i + 1][0], y + h / 2))

    box(ax, (centers[4][0] - 0.1, y + h + 0.55), 1.5, 0.85, "Watermark\nLogo (64x64)", fc="#fdebd0",
ec="#b9770e", fontsize=9.2)
    arrow(ax, (centers[4][0] + 0.6, y + h + 0.55), (centers[4][0] + 0.6, y + h), color="#b9770e")

    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "diagram_embedding.png"), dpi=170)
    plt.close(fig)


def extraction_diagram():
    fig, ax = plt.subplots(figsize=(11, 4.0))
    ax.set_xlim(0, 11); ax.set_ylim(0, 4.0); ax.axis("off")
    ax.set_title("Watermark Extraction Pipeline (non-blind, key-based)", fontsize=13,
fontweight="bold", pad=16)

    steps = [
        "Received Image\n(possibly attacked:\nJPEG / noise / crop /\nrotate / scale ...)",
        "2D DWT\n-> LL sub-band",
        "2-D DCT\n+ SVD -> S_att",
        "Sw' = (S_att - S_orig)\n/ alpha",
        "Reconstruct:\nUw . Sw' . Vw^T",
        "Extracted\nWatermark",
    ]
```

```
    n = len(steps)
    w, h = 1.55, 1.5
    gap = (11 - n * w) / (n + 1)
    x = gap
    y = 0.9
    centers = []
    for s in steps:
        box(ax, (x, y), w, h, s, fontsize=9.2, fc="#eafaf1", ec="#1e8449")
        centers.append((x, x + w))
        x += w + gap
    for i in range(n - 1):
        arrow(ax, (centers[i][1], y + h / 2), (centers[i + 1][0], y + h / 2))

    box(ax, (centers[3][0] - 0.3, y + h + 0.5), 2.0, 0.75,
        "Embedding Key\n{S_orig, Uw, Vw, alpha, shapes}", fc="#f9ebea", ec="#943126", fontsize=8.6)
    arrow(ax, (centers[3][0] + 0.7, y + h + 0.5), (centers[3][0] + 0.7, y + h), color="#943126")

    plt.tight_layout()
    plt.savefig(os.path.join(RESULTS_DIR, "diagram_extraction.png"), dpi=170)
    plt.close(fig)


if __name__ == "__main__":
    embedding_diagram()
    extraction_diagram()
    print("done")
```

Part 5: Tests

Automated tests (pytest).

5.1 conftest.py

Test configuration. 6 lines.

```
import os
import sys

SRC = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", "source_code", "src"))
if SRC not in sys.path:
    sys.path.insert(0, SRC)
```

5.2 test_watermark_core.py

Tests for embedding and extraction. 83 lines.

```
import numpy as np
import pytest

from logo import make_watermark_logo
from metrics import normalized_correlation, psnr
from watermark_core import dct2, embed_watermark, extract_watermark, idct2


@pytest.fixture(scope="module")
def host():
    rng = np.random.default_rng(0)
    from scipy import ndimage
    return np.clip(128 + 40 * ndimage.gaussian_filter(rng.standard_normal((256, 256)), 3) * 6, 0,
255)


@pytest.fixture(scope="module")
def logo():
    return make_watermark_logo(32, "TEST")


def test_dct_roundtrip():
    a = np.random.default_rng(1).random((16, 16))
    assert np.allclose(idct2(dct2(a)), a)


def test_embed_shape_range_and_key(host, logo):
    w, key = embed_watermark(host, logo, alpha=0.1)
    assert w.shape == host.shape
    assert w.min() >= 0 and w.max() <= 255
    assert {"S_original", "Uw", "Vtw", "alpha", "target_shape", "wm_shape"} <= set(key)


def test_clean_roundtrip_recovers_logo(host, logo):
    w, key = embed_watermark(host, logo, alpha=0.1)
    ext = extract_watermark(w, key)
    assert ext.shape == logo.shape
    assert normalized_correlation(logo, ext) > 0.9


def test_imperceptibility_alpha_monotone(host, logo):
    p = [psnr(host, embed_watermark(host, logo, alpha=a)[0]) for a in (0.02, 0.1, 0.4)]
    assert p[0] > p[1] > p[2]


def test_unwatermarked_with_own_key_gives_zero(host, logo):
    _, key = embed_watermark(host, logo, alpha=0.1)
    assert normalized_correlation(logo, extract_watermark(host, key)) == 0.0


def test_odd_sized_host(logo):
    rng = np.random.default_rng(2)
    h = rng.random((101, 77)) * 255
    w, key = embed_watermark(h, logo)
    assert w.shape == h.shape
    assert extract_watermark(w, key).shape == logo.shape
```

```
def test_extract_handles_resized_image(host, logo):
    w, key = embed_watermark(host, logo)
    ext = extract_watermark(w[:200, :200], key)
    assert ext.shape == logo.shape


@pytest.mark.parametrize("bad", [np.zeros((4, 4)), np.zeros((8, 8, 3)), np.full((16, 16), np.nan),
np.zeros(16)])
def test_invalid_host_rejected(bad, logo):
    with pytest.raises((ValueError, TypeError)):
        embed_watermark(bad, logo)


@pytest.mark.parametrize("alpha", [0, -0.1, float("nan")])
def test_invalid_alpha(host, logo, alpha):
    with pytest.raises(ValueError):
        embed_watermark(host, logo, alpha=alpha)


def test_invalid_subband(host, logo):
    with pytest.raises(ValueError):
        embed_watermark(host, logo, subband="XX")


def test_string_array_rejected(logo):
    with pytest.raises(TypeError):
        embed_watermark(np.array([["a"] * 16] * 16), logo)
```

5.3 test_metrics.py

Tests for the metrics. 48 lines.

```
import numpy as np
import pytest

from metrics import (PSNR_CAP_DB, bit_error_rate, mse, normalized_correlation, psnr, ssim,
                     zero_normalized_correlation)


def test_psnr_identical_is_capped():
    a = np.random.default_rng(0).random((32, 32)) * 255
    assert psnr(a, a) == PSNR_CAP_DB


def test_psnr_known_value():
    a = np.zeros((16, 16)); b = np.full((16, 16), 10.0)
    assert mse(a, b) == pytest.approx(100.0)
    assert psnr(a, b) == pytest.approx(10 * np.log10(255 ** 2 / 100.0))


def test_ssim_identical_is_one():
    a = np.random.default_rng(1).random((32, 32)) * 255
    assert ssim(a, a) == pytest.approx(1.0)


def test_nc_basic():
    w = np.zeros((8, 8)); w[2:5, 2:5] = 255
    assert normalized_correlation(w, w) == pytest.approx(1.0)
    assert normalized_correlation(w, np.zeros_like(w)) == 0.0
    assert normalized_correlation(w, np.full_like(w, 255)) == pytest.approx(np.sqrt(9 / 64))


def test_zncc_properties():
    rng = np.random.default_rng(3)
    w = (rng.random((16, 16)) > .8) * 255.0
    assert zero_normalized_correlation(w, w) == pytest.approx(1.0)
    assert zero_normalized_correlation(w, 255 - w) == pytest.approx(-1.0)
    assert zero_normalized_correlation(w, np.full_like(w, 5)) == 0.0
    assert zero_normalized_correlation(w, 0.5 * w + 30) == pytest.approx(1.0)


def test_ber():
    a = np.zeros((4, 4)); b = a.copy(); b[0, :] = 255
    assert bit_error_rate(a, b) == pytest.approx(0.25)
```

```
@pytest.mark.parametrize("fn", [psnr, ssim, normalized_correlation, zero_normalized_correlation,
bit_error_rate, mse])
def test_shape_mismatch_raises(fn):
    with pytest.raises(ValueError):
        fn(np.zeros((8, 8)), np.zeros((9, 9)))
```

5.4 test_attacks.py

Tests for the attacks. 43 lines.

```
import numpy as np
import pytest

from attacks import (ATTACK_NAMES, cropping_attack, jpeg_compression, make_attack_suite,
rotation_attack,
                     scaling_attack)


@pytest.fixture(scope="module")
def img():
    return np.random.default_rng(0).random((128, 128)) * 255


def test_suite_complete_and_shape_preserving(img):
    suite = make_attack_suite(1)
    assert list(suite) == ATTACK_NAMES and len(suite) == 15
    for name, fn in suite.items():
        out = fn(img)
        assert out.shape == img.shape, name
        assert np.isfinite(out).all(), name


def test_stochastic_attacks_are_seeded(img):
    a = make_attack_suite(7)["Gaussian Noise (var=0.02)"](img)
    b = make_attack_suite(7)["Gaussian Noise (var=0.02)"](img)
    c = make_attack_suite(8)["Gaussian Noise (var=0.02)"](img)
    assert np.array_equal(a, b) and not np.array_equal(a, c)


def test_cropping_removes_expected_area():
    out = cropping_attack(np.full((512, 512), 200.0), 0.25)
    kept = (out > 0).mean()
    assert kept == pytest.approx(0.5625, abs=1e-6)


def test_jpeg_quality_bounds(img):
    with pytest.raises(ValueError):
        jpeg_compression(img, 0)
    assert jpeg_compression(img, 90).shape == img.shape


def test_zero_rotation_and_unit_scale_identity(img):
    assert np.allclose(rotation_attack(img, 0), img, atol=1e-6)
    assert np.allclose(scaling_attack(img, 1.0), img, atol=1e-6)
```

5.5 test_image_io.py

Tests for image loading. 66 lines.

```
import numpy as np
import pytest
from PIL import Image

from image_io import ImageLoadError, load_gray_square, save_gray_png, verify_png


def _save(tmp_path, name, im):
    p = tmp_path / name
    im.save(p)
    return str(p)


def test_rgb_wide_image_is_cropped_to_square(tmp_path):
    arr = np.zeros((100, 300, 3), np.uint8); arr[..., 0] = 255
    a, info = load_gray_square(_save(tmp_path, "w.png", Image.fromarray(arr)), 64)
    assert a.shape == (64, 64) and (info.width, info.height) == (300, 100)
    assert a.mean() == pytest.approx(0.299 * 255, abs=1.0)
```

```
def test_rgba_composited_on_white(tmp_path):
    arr = np.zeros((64, 64, 4), np.uint8)
    a, _ = load_gray_square(_save(tmp_path, "t.png", Image.fromarray(arr)), 32)
    assert a.min() > 250


def test_16bit_rescaled(tmp_path):
    arr = (np.arange(64 * 64).reshape(64, 64) * 10).astype(np.uint16)
    a, _ = load_gray_square(_save(tmp_path, "s.png", Image.fromarray(arr)), 32)
    assert 245 <= a.max() <= 255 and 0 <= a.min() <= 10


def test_palette_and_bilevel(tmp_path):
    im = Image.fromarray((np.random.default_rng(0).random((64, 64)) > .5).astype(np.uint8) *
255).convert("1")
    a, _ = load_gray_square(_save(tmp_path, "b.png", im), 32)
    assert a.shape == (32, 32)


def test_missing_empty_corrupt_unsupported(tmp_path):
    with pytest.raises(ImageLoadError):
        load_gray_square(str(tmp_path / "nope.png"))
    e = tmp_path / "empty.png"; e.write_bytes(b"")
    with pytest.raises(ImageLoadError):
        load_gray_square(str(e))
    c = tmp_path / "corrupt.png"; c.write_bytes(b"\x89PNG\r\n\x1a\n" + b"garbage" * 20)
    with pytest.raises(ImageLoadError):
        load_gray_square(str(c))
    t = tmp_path / "doc.txt"; t.write_text("not an image")
    with pytest.raises(ImageLoadError):
        load_gray_square(str(t))


def test_tiny_and_constant_16bit_rejected(tmp_path):
    with pytest.raises(ImageLoadError):
        load_gray_square(_save(tmp_path, "tiny.png", Image.fromarray(np.zeros((8, 8), np.uint8))))
    with pytest.raises(ImageLoadError):
        load_gray_square(_save(tmp_path, "c16.png", Image.fromarray(np.full((64, 64), 500,
np.uint16))))


def test_save_and_verify_roundtrip(tmp_path):
    p = str(tmp_path / "sub" / "x.png")
    save_gray_png(np.full((20, 30), 300.7), p)
    assert verify_png(p, (20, 30))
    assert not verify_png(p, (30, 20))
    assert not verify_png(str(tmp_path / "missing.png"))
    assert np.asarray(Image.open(p)).max() == 255
```

5.6 test_dataset_and_results.py

Checks on the dataset and results. 45 lines.

```
import os

import numpy as np
import pandas as pd
import pytest
from PIL import Image

ROOT = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
META = os.path.join(ROOT, "data", "metadata.csv")
pytestmark = pytest.mark.skipif(not os.path.exists(META), reason="dataset not built")


@pytest.fixture(scope="module")
def meta():
    return pd.read_csv(META)


def test_exactly_100_unique_images(meta):
    assert len(meta) == 100
    assert meta.image_id.is_unique and meta.key.is_unique
    hashes = []
    for iid in meta.image_id:
        a = np.asarray(Image.open(os.path.join(ROOT, "data", "original", iid + ".png")))
```

```
        assert a.shape == (512, 512) and a.dtype == np.uint8
        hashes.append(a.tobytes().__hash__())
    assert len(set(hashes)) == 100


def test_synthetic_labelled(meta):
    assert set(meta[meta.is_synthetic == 1].category) == {"synthetic_control"}
    assert (meta[meta.is_synthetic == 0].category != "synthetic_control").all()
    assert meta[meta.is_synthetic == 0].source_sha256.str.len().eq(64).all()


@pytest.mark.skipif(not os.path.exists(os.path.join(ROOT, "results", "results_attacks.csv")),
reason="no results")
def test_batch_completeness(meta):
    att = pd.read_csv(os.path.join(ROOT, "results", "results_attacks.csv"))
    assert len(att) == 100 * 15 * 2
    assert (att.status == "ok").all()
    assert att.groupby(["image_id", "protocol"]).size().eq(15).all()
    assert att.nc.between(0, 1).all() and att.ber.between(0, 1).all()
    imgs = pd.read_csv(os.path.join(ROOT, "results", "results_images.csv"))
    assert imgs.output_valid.all() and (imgs.status == "ok").all()
    for iid in meta.image_id:
        assert os.path.exists(os.path.join(ROOT, "outputs", "watermarked", iid + "_wm.png"))
```

Part 6: Configuration

Settings and dependencies.

6.1 config.json

Experiment settings. 37 lines.

```
{
  "alpha": 0.1,
  "image_size": 512,
  "logo_size": 64,
  "logo_text": "CYBERION",
  "other_logos": [
    "OTHERLOGO",
    "OWNER-B"
  ],
  "protocols": [
    "uint8",
    "float"
  ],
  "primary_protocol": "uint8",
  "base_seed": 20260713,
  "alphas": [
    0.02,
    0.05,
    0.08,
    0.1,
    0.15,
    0.2,
    0.3,
    0.4
  ],
  "sweep_attacks": [
    "JPEG Q=50",
    "Gaussian Noise (var=0.02)",
    "Gaussian Blur (sigma=1.5)",
    "Rotation 5 deg"
  ],
  "n_controls_per_image": 5,
  "nc_threshold": 0.75,
  "data_dir": "../../data",
  "results_dir": "../../results",
  "outputs_dir": "../../outputs"
}
```

6.2 requirements.txt

Python packages used. 13 lines.

```
numpy==2.5.3
scipy==1.18.1
scikit-image==0.26.0
PyWavelets>=1.8
Pillow==12.3.0
pandas==3.0.5
matplotlib==3.11.2
opencv-python-headless==5.0.0.93
blind-watermark==0.4.4
imagehash==4.3.2
python-docx==1.2.0
python-pptx==1.0.2
pytest>=8
```